

User Manual

XY Phase Series

Spatial Light Modulators

With PCIe Controller



Information furnished in this guide is believed to be accurate and reliable. However, no responsibility is assumed for its use, or for any patent infringements or other rights of third parties that may result from its use.

Meadowlark Optics, Inc.
5964 Iris Parkway
Frederick, CO 80530
USA

Telephone: (303) 833-4333
Fax: (303) 833-4335
sales@Meadowlark.com
www.Meadowlark.com

Table of Contents

1. INTRODUCTION	1
1.1 Spatial Light Modulator Principles	1
1.2 Phase Modulation	2
1.3 Optical Test Setup	3
2 SYSTEM SETUP.....	7
2.1 Minimum System Requirements.....	7
2.2 Software Installation Instructions	8
2.3 Hardware Installation Instructions.....	8
2.4 Device Driver Installation	10
3 SOFTWARE OPERATION	11
3.1 Sequences.....	12
3.2 Running a sequence	12
3.3 Opening new sequence list	13
3.4 SLM Temperature.....	13
3.5 LUT and Wavefront Calibrations Button	13
3.6 Triggering.....	14
3.7 Pattern Generation.....	15
4 SOFTWARE DEVELOPMENT KIT INTRODUCTION.....	22
4.1 Modes of Operation	22
4.2 Function Summary and Usage.....	22
4.3 Function Definitions	23
4.4 Optional Functions	28
4.5 Optional Image Generation Functions	29
5 GENERATING A CUSTOM LUT CALIBRATION.....	37
5.1 Background.....	37
5.2 Introduction to the Diffractive Calibration Method.....	37
5.3 Collecting and Post Processing Static measurements.....	39
5.4 Dynamic Calibration of LUT Files.....	40
5 MECHANICAL HOUSING	44

6	SLM CARE AND CLEANING	45
7	TROUBLESHOOTING	46
7.1	Image appears incorrect.....	46
7.2	LEDs on the 1920x1152 or 1024x1024 PCIe hardware are blinking	46

List of Figures

Figure 1 Cross sectional illustration of a liquid crystal spatial light modulator.	1
Figure 2 This diagram illustrates the liquid crystal orientation with respect to the coverglass and VLSI backplane as a function of applied voltage. The molecules are parallel to the coverglass and backplane when no field is applied, and are nearly perpendicular to the coverglass and backplane when full field is applied.	2
Figure 3 A Twyman-Green interferometer for testing an XY Phase SLM.	4
Figure 4 Off-axis optical setup for the XY Phase Series SLMs.	6
Figure 5 Complete SLM system with all associated cabling (left) 1920x1152, (right) 1024x1024.	7
Figure 6 Electrical connection diagram.	8
Figure 7 The One Stop Board (left) and its installation into a computer workstation (right).	9
Figure 8 The Device Manager when the software is properly installed.	10
Figure 9 Device Manager prior to rebooting	11
Figure 10 Main window for the Meadowlark Blink PCIe software	12
Figure 11 Timing for the 1024 x 1024 pixel SLM with external triggers are enabled and output pulses enabled. The frame rate of image loads is driven by the frame rate of the external hardware trigger. When the falling edge of an external trigger arrives (shown in green) the hardware will initiate an image update on the SLM. An output pulse is generated to acknowledge receipt of the trigger (shown in blue). If flip immediate is disabled the output pulse will occur up to 0.696 ms after the falling edge of the external trigger is received. If flip immediate is enabled the image will update immediately after the falling edge of the trigger is received. After the output pulse is generated the image on the SLM will be updated. In this example a focal point is toggled on and off a detector showing the optical response of the SLM receiving a new image (shown in yellow).	15
Figure 12 Bessel GUI	16
Figure 13 Optical Layout for BEST volumetric imaging	17
Figure 14 BEST Bessel GUI	18
Figure 15 Grating GUI	18
Figure 16 Fresnel Lens GUI	19
Figure 17 Hologram Generation GUI	20
Figure 18 Solid, Stripe, Random Phase, Checkerboard GUI	20
Figure 19 Superimpose Images GUI.	21
Figure 20 Zernike Polynomial GUI	21
Figure 21: Function Call Order of Operations	22
Figure 22: LabVIEW Call Library Function Node to interface to the Meadowlark Optics DLL	23
Figure 23 Generating an Affine Calibration	33
Figure 24 Typical optical setup for diffractive calibration	38
Figure 25 Typical diffractive calibration data.	38

Figure 26 Simulated Fourier Plane with stripe patterns	39
Figure 27 National Instruments Measurement and Automation Explorer used to read from your DAQ board.	41
Figure 28 Diffractive LUT GUI, shown with PCIe SLM attached for dynamic LUT generation.	41
Figure 29 Use LUT Offset to shift the voltage range of the LUT to maximize phase stability.	42
Figure 30 Validating a calibrated LUT. The phase vs. voltage should be linear from 0 to 2π	43
Figure 31 Front and side views of 1920 SLM Optical Head. Dimensions are in inches.	44
Figure 32 Front and side views of 1024 SLM Optical Head. Dimensions are in inches.	44

1. Introduction

This User Manual covers the 1024 x 1024, and 1920 x 1152 XY Phase Series of liquid crystal on silicon spatial light modulators (SLMs) now available from Meadowlark Optics. The manual instructs users on how to set up the PCIe electronic hardware, optics, and operate the system software. SLM system setup is complex, incorporating optical components, mounting and positioning hardware, custom drive electronics hardware, and a proprietary software interface. We strongly recommend that all users first read and familiarize themselves with the entire User Manual before initiating the setup and start up procedures.

1.1 Spatial Light Modulator Principles

A SLM is a device that modulates light according to a fixed spatial (pixel) pattern. The XY Phase Series SLMs convert digitized data into coherent optical information appropriate for a wide variety of applications, including beam steering, optical tweezers, diffractive optics, pulse shaping, and more. These applications require modulators that can easily and rapidly change the wavefront of a coherent beam. By combining the electro-optical performance characteristics of liquid crystal materials with silicon-based digital circuitry, Meadowlark Optics now offers high resolution SLMs that are also physically compact and optically efficient.

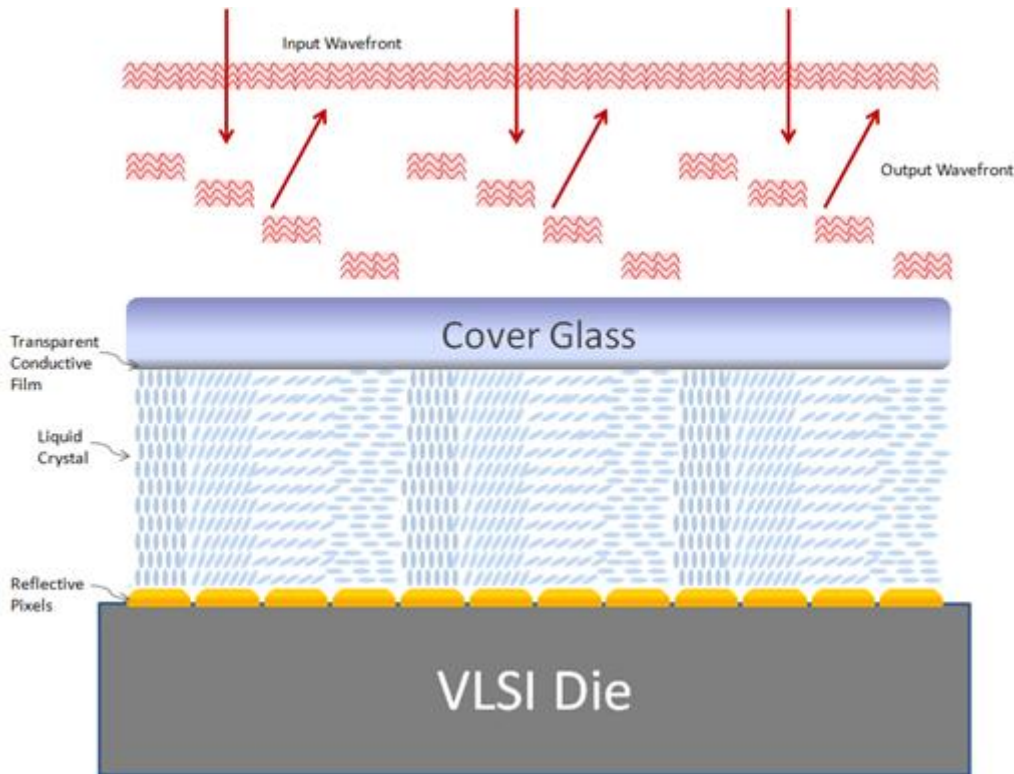


Figure 1 Cross sectional illustration of a liquid crystal spatial light modulator.

Figure 1 shows the cross section of an LC SLM. Polarized light enters the device from the top, passes through the cover glass, transparent conductive film, and liquid crystal layer. Light is reflected off the pixel electrodes and returns on the same path. Drive signals from the hardware interface independently drive each pixel on the SLM. The analog voltage applied to each pixel produces an electric field between that pixel and the cover glass, resulting in a change in the optical properties of the LC layer. Because each pixel is independently controlled, a phase pattern may be generated by loading different voltages onto each pixel.

1.2 Phase Modulation

The XY Phase Series Spatial Light Modulators are fabricated with nematic liquid crystal, aligned in a homogenous configuration. Nematic liquid crystal has a variable electro-optic response to voltage. Fig. 2 shows a simplified side view of a spatial light modulator's liquid crystal layer. When no voltage is applied to the LC, the molecules are parallel to the SLM coverglass and VLSI backplane. In this case, incident light will experience the largest difference between the extraordinary (n_e) and ordinary (n_o) index of refraction. As the voltage applied to the liquid crystal is increased, the LC will tilt until the extreme is reached and the LC is nearly normal to the SLM coverglass and VLSI backplane. At this point the difference between the extraordinary and ordinary index of refraction is nearly zero.

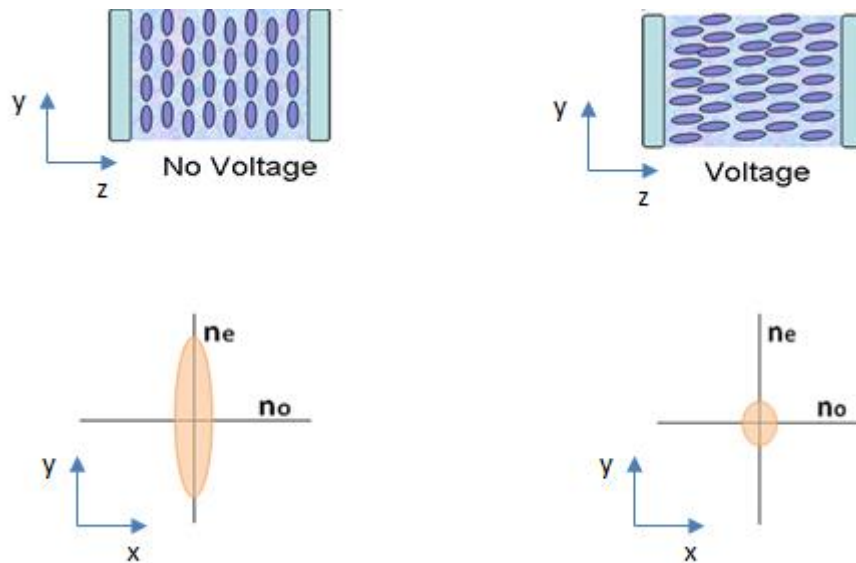


Figure 2 This diagram illustrates the liquid crystal orientation with respect to the coverglass and VLSI backplane as a function of applied voltage. The molecules are parallel to the coverglass and backplane when no field is applied, and are nearly perpendicular to the coverglass and backplane when full field is applied.

If light incident upon the SLM is linearly polarized parallel to the extraordinary axis, then a pure, voltage dependent phase shift will be observed. For example, if no voltage is applied to the pixel, the maximum phase retardation (typically a full wave at the design wavelength) will be applied. Likewise, if the pixel is programmed for maximum voltage, a minimum phase delay is applied. The result is a programmable phase modulation on a per pixel basis.

Each of the SLM pixels is independently programmable to 256 discrete voltage states, all providing phase modulation. The response of the liquid crystal within the SLM to the applied pixel value (voltage) is nonlinear. To account for this, each SLM is shipped with a custom look-up-table (LUT). When this LUT is applied to an input image, the result is a linear output phase response ranging from 0 to 2π .

1.3 Optical Test Setup

Depending on the application of the XY Phase Series SLM, many different optical setups can be used. Two examples of phase-only optical test setups are shown below.

The first optical setup, illustrated in Figure 3, is a modification of a Twyman-Green interferometer (Handbook of Optics. Vol. 1, pp. 2.28-2.29). Here a monochromatic, collimated light source (i.e. laser beam expanded so it is larger than the diagonal of the SLM) passes through a non-polarizing beamsplitter such that the beam is divided into two beams, with nearly equal intensity. One of these beams illuminates the XY Phase Series SLM, while the other illuminates a reference mirror. Each of these reflected beams is then recombined at the image plane of a lens. If the reference mirror and the SLM are carefully aligned such that they are nearly coplanar, interference fringes will be visible at the image plane. A camera is placed at the image plane with proper magnification for easier viewing of the active area of the SLM and the interference fringes. When the XY Phase Series SLM is driven with different phase patterns, the corresponding interference fringes can be viewed. Analyzing the interference fringes will then provide insight into the phase modulation provided by the XY Phase SLM.

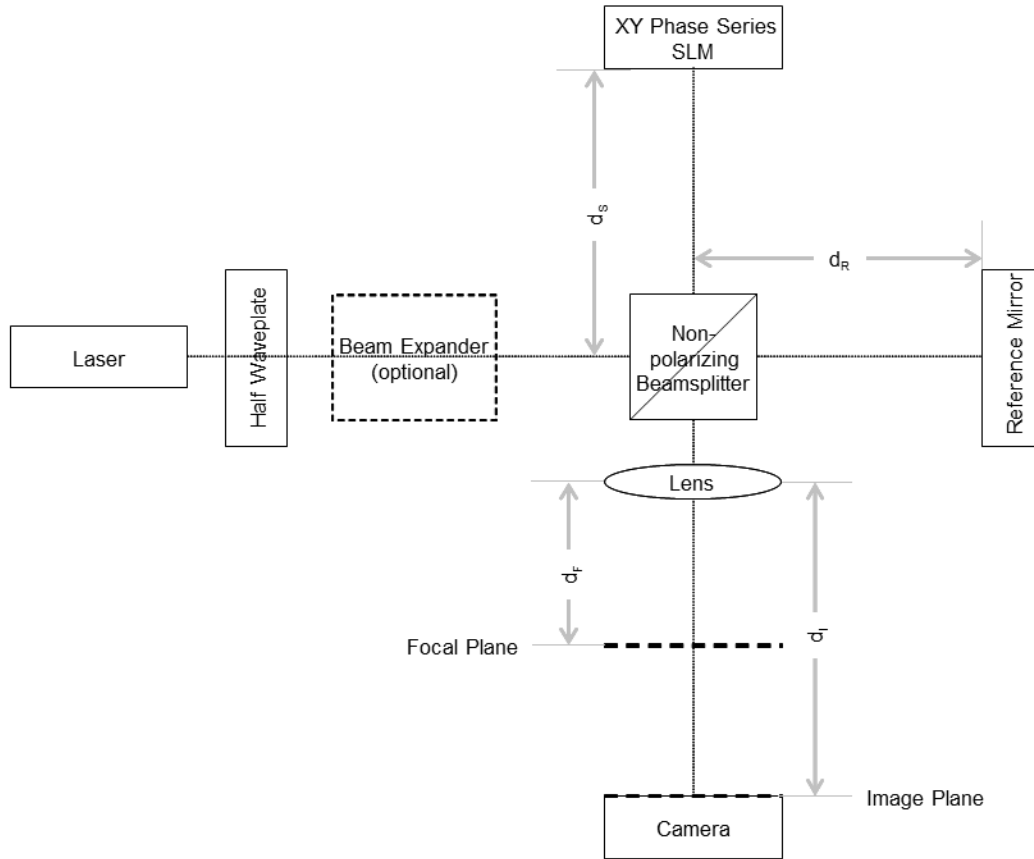


Figure 3 A Twyman-Green interferometer for testing an XY Phase SLM.

To ease the alignment of the reference mirror and the XY Phase Series SLM in the Twyman-Green interferometer, place the camera or a card into the beam path at the focal plane of the lens. There should be two spots visible, one from the reference mirror and one from the SLM (there may also be faint high-order diffraction spots from the SLM). If utilizing a beamsplitter cube, there are likely to be additional spots that are ghost images from the beamsplitter. When adjusting the tip/tilt of either the reference mirror or the SLM, the ghost spots will not move, but the true spots from the reference mirror and the SLM will move. Once the correct two spots are located, adjust the tip and tilt of the reference mirror and/or the SLM until these two spots are aligned on top of one another. Place the camera back in the image plane, or remove the card, and interference fringes should be visible in the image plane. By slowly adjusting the tip and/or tilt of just the reference mirror or SLM, the desired interference fringes should be readily obtained.

If working with a laser diode it is important to note that the coherence length of laser diodes are typically much shorter than the coherence length of a laser. With a Twyman-Green interferometer it is critical that both the SLM leg and the reference mirror leg have the same optical path length, ensuring that the short coherence length of a laser diode does not significantly reduce the visibility of the interference fringes. To ensure equal path lengths, set the camera such that it is focused onto the SLM – distinct features of the silicon backplane, such as individual pixels

or the edge of the active area, should become sharp. Blocking the reference leg beam will likely be necessary to prevent crosstalk in the camera. Then move the block from the reference leg to the SLM leg and move the reference mirror closer to, or further from, the beam splitter until it is also in sharp focus on the camera. Since the reference mirror will likely not have any features with which to see the best focal point, look for focus of dust particles, or perhaps very carefully place a card onto the reference mirror such that it covers about half of the beam. The card edge can then be used as a guide to find the best focal position of the reference mirror.

There are a few important issues to remember whenever working with an SLM.

1. **Polarization** – The SLM is basically a variable single-order retardation plate, or wave plate. Like all wave plates, there is a fast axis and a slow axis. However, with the XY Phase Series SLMs the index of refraction along the slow axis can be decreased electronically. **While the liquid crystal alignment is occasionally customized, in general on the 1920 x 1152 the slow axis is vertical (parallel to the short axis of the SLM), and on the 1024 x 1024 the slow axis is horizontal.**

When the light source is linearly polarized and parallel to the slow axis of the SLM, the result is phase-only modulation of the light source. If the orientation of the incident polarization is incorrect, there would be no modulation observed with variable voltage. Placing a half wave plate between the laser and the beamsplitter will greatly facilitate achieving the desired polarization alignment.

If there is any question about the laser producing linearly polarized light, place a polarizer between the laser and the half wave plate. This will ensure that the output of the laser is linearly polarized, and the half wave plate can then be used to rotate the angle of the polarization.

2. **Diffraction** – The SLM has discrete, reflective pixel pads that diffract light into higher orders in addition to the 0th order. This diffraction can easily be seen in the focal plane of a lens. There will be a very bright center spot (0th-order), surrounded by a grid of spots becoming progressively dimmer as they get further from the 0th-order.
3. **Dispersion** – Liquid crystal devices are not achromatic due to the nonlinear change in index of refraction as a function of wavelength. This dispersion means that a device designed to provide a 2π phase stroke at one wavelength, will provide less than 2π phase stroke at a longer wavelength, and more than 2π phase stroke at a shorter wavelength. A wavelength dependent calibration is provided with the SLM such that input graylevels of 0 to 255 linearly map to phase delays of 0 to 2π . If the wavelength is changed it will be necessary to re-calibrate the SLM.
4. **Optical Quality** – Due to the very small pixel pitch of the SLM, it is important to use high quality optics. A single element lens will generally have too much spherical aberration to

provide a good, sharp focus across the entire clear aperture of the SLM. As a result, it is recommended to always use at least a doublet lens. For the same reason, the longer the focal length of the lens, the better the resulting image at the image plane. In addition, the transmitted wavefront distortion of most beamsplitter cubes is typically $\sim\lambda/4$, contributing to aberrations at the image plane. The use of a beamsplitter plate in place of a beamsplitter cube could reduce aberrations but using a beamsplitter plate requires the use of a compensating plate in the reference leg of the Twyman-Green interferometer.

An off-axis setup, shown in Figure 4, is designed to maximize throughput by eliminating the non-polarizing beam splitter. A laser beam is incident on the SLM at a slight angle reflects off the reflective pixel pads, and then is imaged with a lens onto a camera. In this setup it is most common to put the camera at the focal plane of the lens. The SLM can be used to modulate the 0th order or can be used as a diffractive optic manipulating the 1st order.

The off-axis angle should be kept to a minimum to reduce crosstalk effects due to the beam traveling through more than one pixel region. Minimizing the off-axis angle also keeps the phase stroke closer to the designed value. We typically recommend an angle of incidence of 15 degrees or less. If a wider angle is used the SLM should be recalibrated.

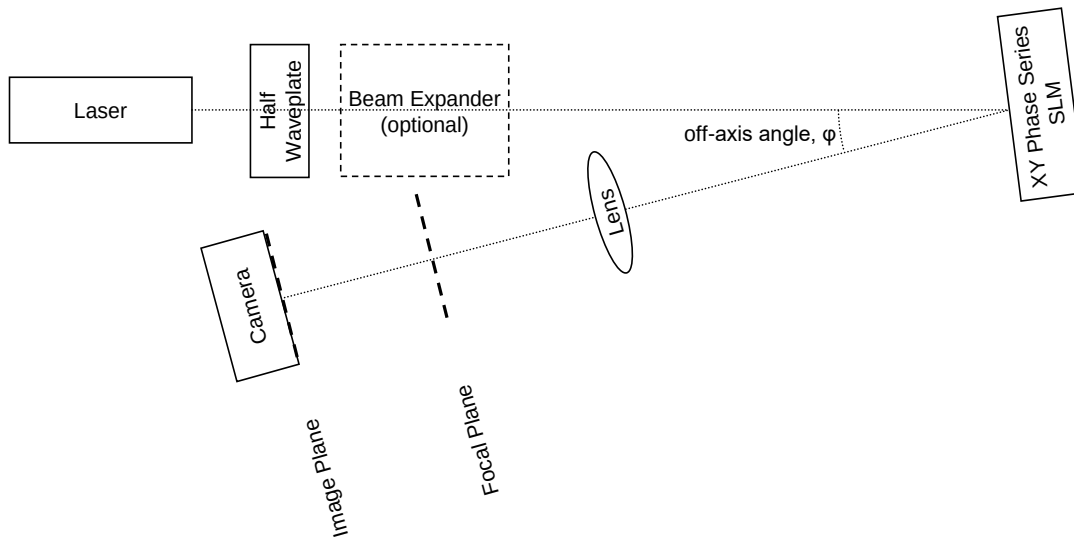


Figure 4 Off-axis optical setup for the XY Phase Series SLMs.

2 System Setup

In Figure 5 the complete 1920 x 1152 SLM system and 1024 x 1024 SLM system are shown. Both systems are driven through a x8 PCIe interface. This system consists of a PCIe Driver mounted to the SLM optical head, an external power cable, a x8 PCIe cable, a x8 PCIe host adapter card, and a USB card that contains software, documentation, and user manual. The power cable must be connected from the controller to a power strip, the PCIe cable must be connected from the host adapter card which is mounted in a x8 or greater Gen3 PCIe slot in the computer to the PCIe Driver.



Figure 5 Complete SLM system with all associated cabling (left) 1920x1152, (right) 1024x1024.

To set up your SLM begin by verifying your computer meets the minimum recommended system requirements outlined in section 2.1. The SLM software installation is outlined in section 2.2. After the software is installed, shut down your computer and install the SLM hardware. When your computer is re-booted, check the device manager as outlined in section 2.3 to validate that the device driver was properly installed.

2.1 Minimum System Requirements

To effectively utilize your SLM system some basic computing hardware is required as outlined by the recommended system requirements listed below.

- 1 GHz dual-core processor or better
- 2 GB RAM or better
- Available x8 (or greater) PCIe slot for the 1920 x 1152 or 1024 x 1024 SLM.
- 500 MB available disk space
- Windows®-based computer operating system (Windows 10® 64-bit)
- **Nvidia GPU supporting OpenCL 1.2 or higher is required**

2.2 Software Installation Instructions

The Meadowlark software on the USB flash drive contains the executable code necessary to operate the SLM, as well as sample pattern files, calibrations, software development kits, documentation, and various other support files. Please follow these steps to install the software.

1. Insert the USB flash drive that came with your SLM system into the USB drive.
2. Copy the *.msi, Preferences.ini, your custom LUT file, and your custom calibration file to your computer. Double click on the *.msi file to launch the installer.
3. Follow the onscreen instructions to complete the installation.
4. When the installation is complete, eject the USB drive from the drive and store it away carefully.
5. After installation is complete, the software should be ready to use. A shortcut to the executable will be installed on the desktop, and in the Start Menu.
6. The software will be installed under C:\Program Files\Meadowlark Optics\Blink Plus. The software development kit documentation and example programs can be found at C:\Program Files\Meadowlark Optics\Blink Plus\SDK.

2.3 Hardware Installation Instructions

Figure 6 shows the cabling required for the electrical system. It is assumed that the customer has a computer workstation with a free PCIe x8 (or greater) slot available for the 1920 x 1152 or 1024 x 1024 SLM. The computer should be running Windows® 10 64 bit. It is also useful to have a small, surge protected power strip available.

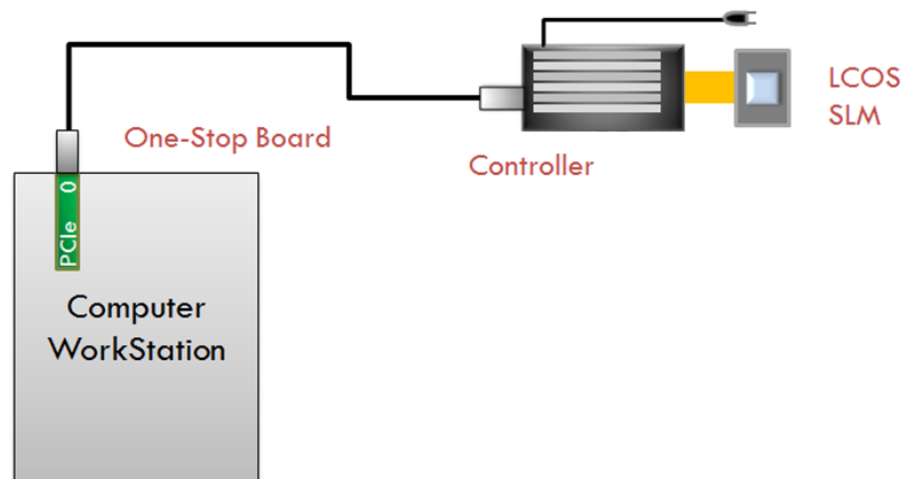


Figure 6 Electrical connection diagram.

To setup the electrical system:

- 1) Power down the computer workstation
- 2) Open the computer and install the PCIe Host adapter board in a spare PCIe slot on the computer's motherboard. Make sure the board is seated properly in the slot and secure the board's panel bracket to the computer with a screw.

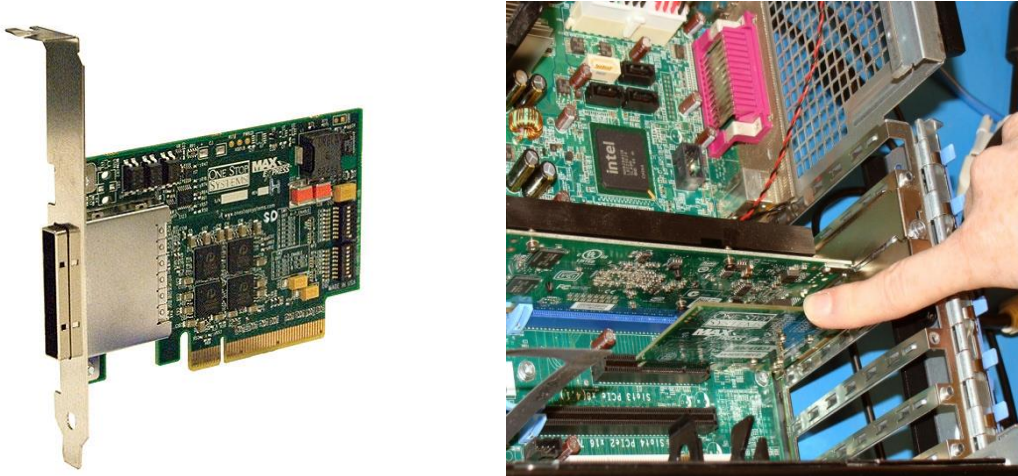


Figure 7 *The One Stop Board (left) and its installation into a computer workstation (right).*

- 3) Connect the PCIe cables between the controller box and the One-Stop Board in the Computer Workstation.



- 4) Connect the power cable to the controller box.
- 5) Boot the computer. The default standard for PCIe hardware is that communication between the computer and hardware is established during the boot cycle. So it is important that the SLM controller is powered prior to powering the computer.

2.4 Device Driver Installation

After the software has been installed, the hardware has been installed, and the computer has been rebooted installation of the device driver should be automatically set up. However, if there seems to be a problem with the system, verifying that the device drivers is a good place to start. Open Device Manager and look for “Meadowlark” as shown in Figure 8. There should be one sub-item labeled “Meadowlark Device (SLMP1024)” for the 1024 x 1024 SLM, or “Meadowlark Device (SLMP1920)” for the 1920 x 1152 SLM.

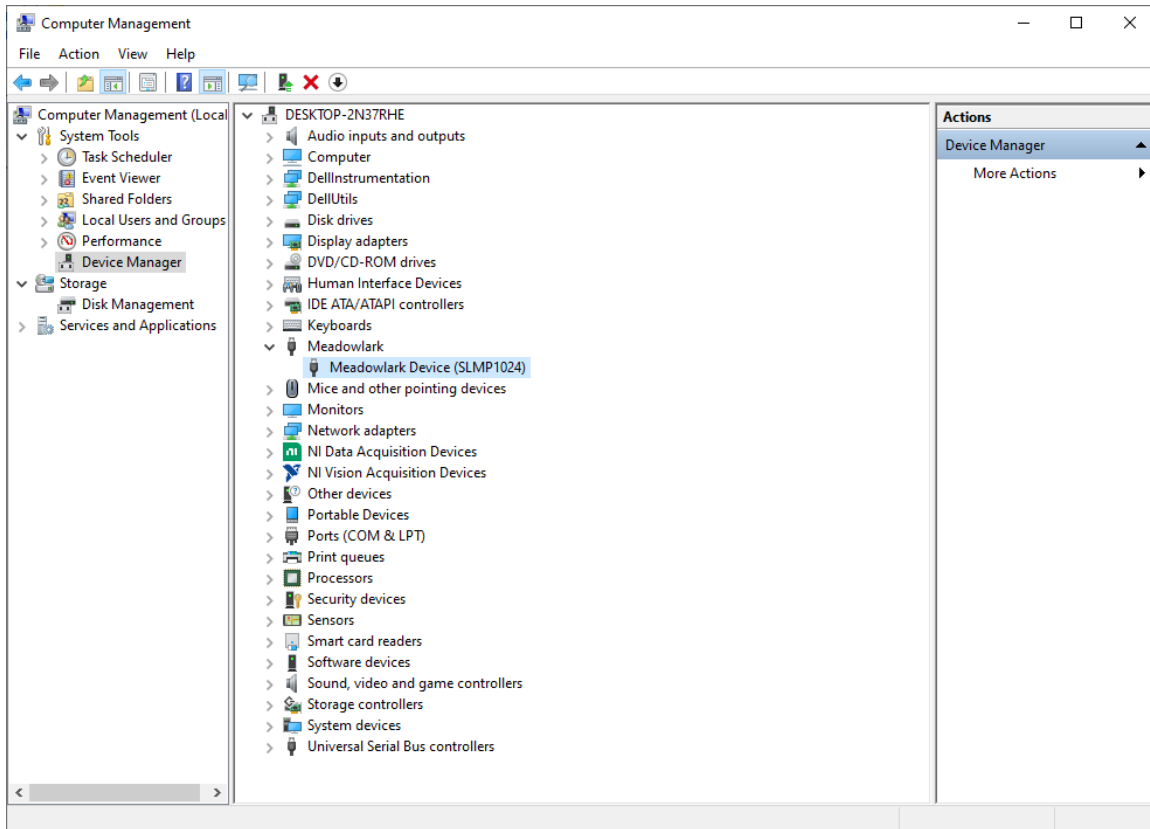


Figure 8 The Device Manager when the software is properly installed.

Note: The PCIe SLM driver will only be visible after installing the hardware and software and rebooting the computer. If, after rebooting, the drivers still do not appear, look under System Devices for a PCI Standard RAM Controller or under Other Devices for a PCI Memory Device as shown in Figure 9.

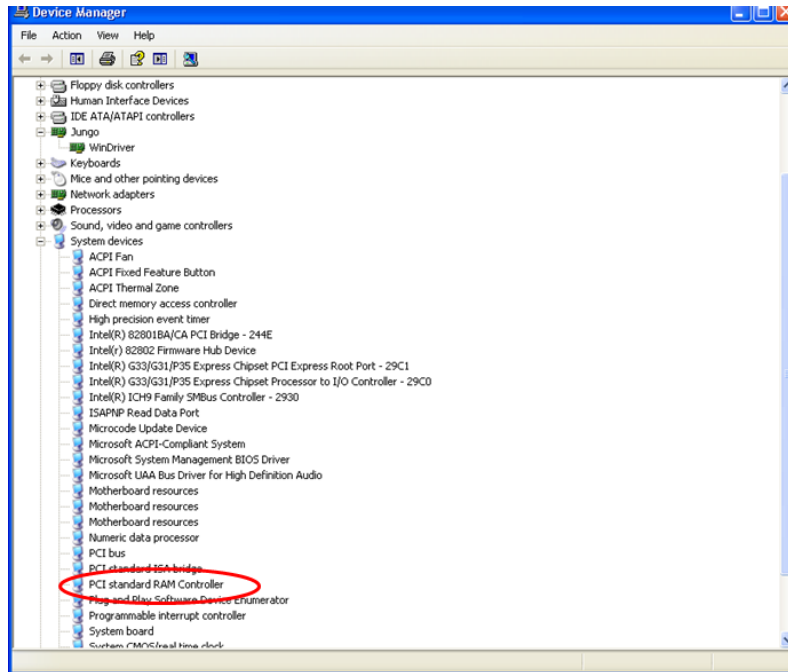


Figure 9 Device Manager prior to rebooting

If rebooting does not update the 'PCI Standard RAM Controller' or 'PCI Memory Device' driver, then right click on this item and select to update the driver. In the Hardware Update Wizard at the prompt that says, 'Can Windows connect to Windows Update to search for software', select 'No, not this time'. At the next window, select to 'Install from a list or specific location'. At the next window, select 'Don't search. I will choose the driver to install'. At the next window within the Model list box the SLM driver should be found. Click on it and proceed with the driver update. If it is not found, click on the background of the list box to un-select all items, and then click on the 'Have Disk' button. This will allow you to browse to C:\Program Files\Meadowlark Optics\Blink Plus. If it is still not possible to install the driver contact Meadowlark Support for further options (slmsupport@meadowlark.com).

3 Software Operation

To start the software, double click the "BlinkPlus" shortcut located on your desktop, or click on the Windows "Start" button, select, "Run...", type "C:\Program Files\Meadowlark Optics\Blink Plus\BlinkPlus.exe", then click "OK". When the software is started, the screen shown in Figure 10 should be visible.

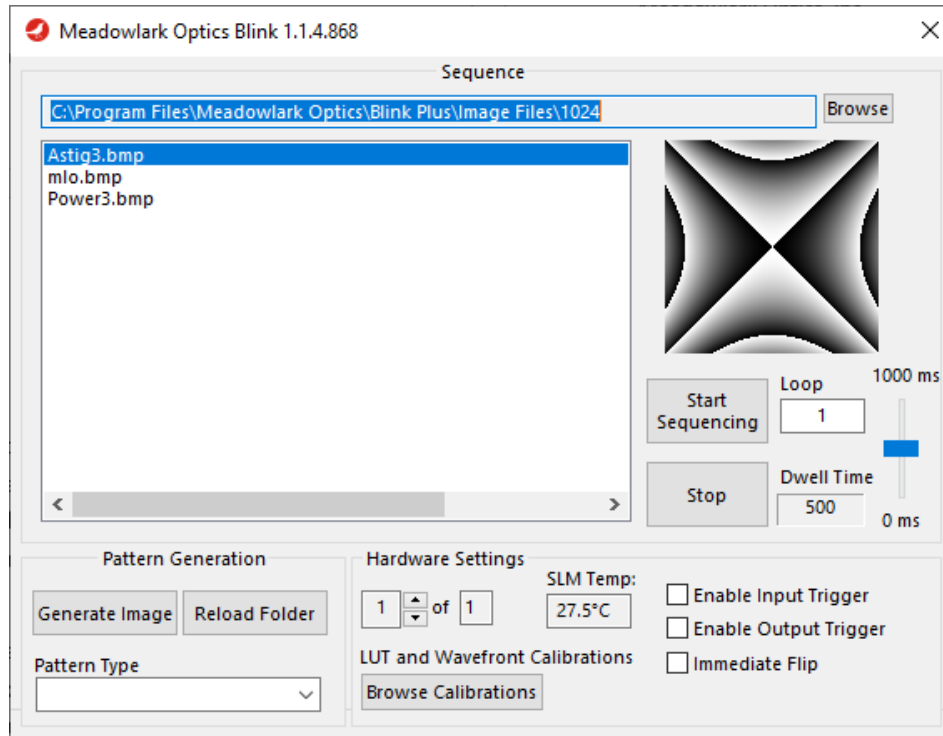


Figure 10 Main window for the Meadowlark Blink PCIe software

The SLM will be powered automatically when the computer boots. After the software has opened, the Pattern File selected in the Sequence List should appear both on the GUI and on the SLM.

3.1 Sequences

The Sequence list is defined by reading the contents of a folder and searching for bitmaps. The software stores information about the last folder used and loads the images from this folder upon initialization. Note that the images are not sorted, so to force a series of images to occur in a specific order it is recommended file names within a folder follow a numbering sequence such as: 000Img.bmp, 001Img.bmp, 002Img.bmp...etc.

To change the Pattern File currently loaded into the SLM, simply point the mouse at a different Pattern File in the Sequence List and click. This new Pattern File should now be highlighted and be immediately loaded into the SLM. The cursor key (↑ and ↓) can also be used to select different Pattern Files in the Sequence list.

3.2 Running a sequence

The Pattern Files can be changed repetitively by clicking the “START” button located on the bottom of the main window. Once the “START” button has been pressed, the system will sequence through all the Pattern Files in the current Sequence List at a user defined frame rate for a user specified number of loops. Note that the frame rate slider can be adjusted in 1 ms steps by clicking on the dwell time slider and using the ↑ and ↓ to adjust the frame rate. The dwell time is only used if external triggers are disabled. This is a software-based

timer that sets a dwell time delay between image writes to hold the image on the SLM for a user-defined period. The timer does not account for time to complete an image write and will be limited to the precision of software-based timers.

3.3 Opening new sequence list

To open a new sequence file, move the cursor over the “Browse...” button and click. This opens the standard Windows wizard for opening a file or folder. The user is asked to locate a folder on the hard disk. Once located, the new sequence is automatically loaded into the system with the images currently contained within the selected folder. If the images within the folder are moved or deleted after loading into the system, it is necessary to click the “Browse...” button again and reselect the folder.

3.4 SLM Temperature

The SLM software will periodically poll the current SLM temperature. The liquid crystal will lose modulation depth as the SLM heats. If the SLM is used with a high-power laser it is recommended that the SLM be calibrated at the desired operating temperature.

3.5 LUT and Wavefront Calibrations Button

This button opens a dialog that allows the user to browse for a LUT calibration, browse for a wavefront correction, and adjust the coverglass voltage **ONLY** if recommended by a Meadowlark Optics Employee.

3.5.1 Look Up Table (LUT) Information

Of primary interest to the user is the ability to change the LUT, or “look up table”. A LUT is used to map input image data that ranges from 0 to 255 to a set of new output values that result in a linear from 0 to 2π phase shift. The software supports several LUT file formats: *.LUT, and *.txt. A *.LUT file contains two columns: the first column in the input graylevel value (0 – 255), and the second column in the output value of 0 to 2047 for the 1920 x 1152, and 0 to 4096 for the 1024 x 1024. A *.txt calibration file is a regional calibration. To ensure proper formatting, the regional calibration file should only be generated using Meadowlark Optics calibration tools, which are described in greater detail in Section 6.

Each SLM system is shipped with a custom LUT designed to enable a linear phase response versus pixel value. This LUT will be named “slmXXXX_atYYY.lut”, where XXXX corresponds to the serial number of the SLM, and YYY corresponds to the wavelength the calibration was generated at. When using this LUT, a linear progression of input 0 to 255 data should result in a linear output phase of 0 to 2π .

3.5.2 Wavefront Correction File

This is an aberration correction image that is superimposed with each user defined phase pattern by adding the two images together modulo 256, and then processing the final image through the LUT calibration. This is used to remove aberrations from the SLM, or from the SLM and optical train if calibrated in the customer’s optical system.

3.5.3 Coverglass Voltage

It is generally not recommended that the user edit the coverglass voltage because improper settings could cause permanent damage to the SLM. The proper default will be set by Meadowlark Optics. However, if instructed by a Meadowlark Optics employee this value can be edited.



3.6 Triggering

The user can either control the rate at which Blink sequences using software timers or using external triggers. Blink will remember the last state used as the default.

For the 1920 x 1152 and 1024 x 1024 SLM the input and output triggers can be accessed through SMA connectors at the bottom of the PCIe controller. If the output pulse on image flip checkbox is enabled, then an output pulse is generated when the dual port memory bank on the PCIe hardware flips.

On the 1920 x 1152 SLM the new image will begin loading on the SLM 1.18 ms after the output pulse is generated. If input triggers are enabled, then the falling edge of an input trigger will control the rate at which images are updated on the SLM. An output pulse will be generated to acknowledge a received trigger and will occur up to but not in excess of 1.18 ms of receiving an external trigger falling edge. The image on the SLM will update 1.18 ms after the output pulse is generated.

On the 1024 x 1024 SLM the new image will begin loading on the SLM after the output pulse is generated. If input triggers are enabled, then the falling edge of an input trigger will control the rate at which images are updated on the SLM. An output pulse will be generated to acknowledge a received trigger and will occur up to but not in excess of 0.696 ms of receiving an external trigger falling edge. On the 1024 x 1024 SLM the user can optionally enable flip immediate mode, which interrupts an image refresh and allows the dual port memory bank to flip and an output pulse to be generated immediately after receipt of the falling edge of an input trigger. The image on the SLM will update 0.696 ms after the output pulse is generated. This timing is illustrated in Figure 11.

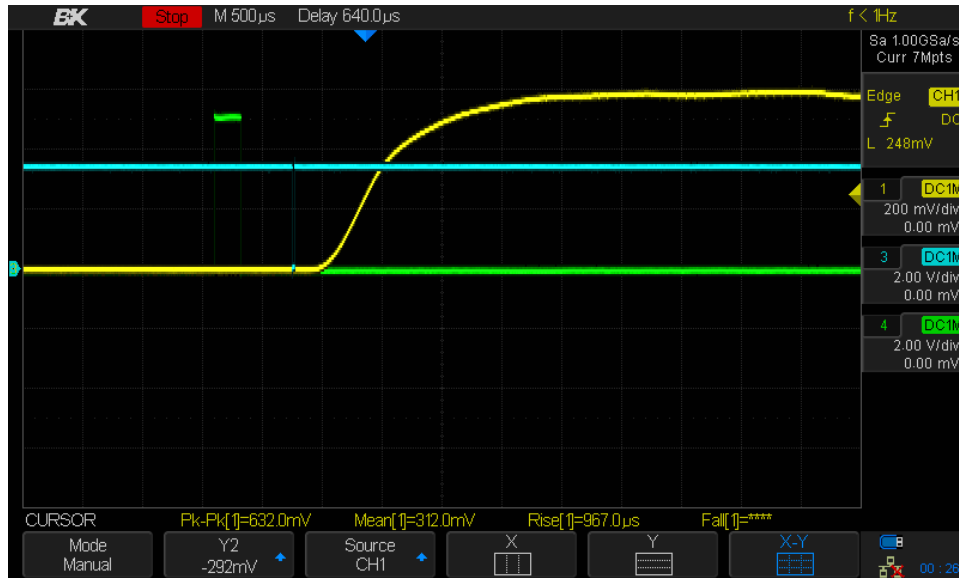


Figure 11 Timing for the 1024 x 1024 pixel SLM with external triggers are enabled and output pulses enabled. The frame rate of image loads is driven by the frame rate of the external hardware trigger. When the falling edge of an external trigger arrives (shown in green) the hardware will initiate an image update on the SLM. An output pulse is generated to acknowledge receipt of the trigger (shown in blue). If flip immediate is disabled the output pulse will occur up to 0.696 ms after the falling edge of the external trigger is received. If flip immediate is enabled the image will update immediately after the falling edge of the trigger is received. After the output pulse is generated the image on the SLM will be updated. In this example a focal point is toggled on and off a detector showing the optical response of the SLM receiving a new image (shown in yellow).

3.7 Pattern Generation

The software allows users to dynamically generate images to load to the SLM. The supported modes include solids, stripes, checkerboards, random phases, blazed gratings, sinusoidal gratings, Bessel beams (spiral phase, forked phase, axicons, concentric rings), holograms, Zernike polynomials, Fresnel lenses (cylindrical and circular), and superimposed images. Image generation capabilities are supported both through the main GUI, and through the software development kits by loading the Meadowlark Optics Image Generation DLL (see Section 7.5 for further information).

To generate an image, select the image type from the Pattern Generation drop box, and then click “Generate Image”. This will bring up one of the following dialogs. As the image parameters are edited, the SLM automatically updates with the current image data. The image can be saved to a user defined folder (the current folder is the default), and by clicking the “Reload Folder” button the images in the current sequence list will be automatically updated to include the newly generated image.

3.7.1 Bessel Beam

The Bessel Beam GUI (Figure 12) allows for generation of vortex beams, axicons, and concentric ring patterns. As the image parameters are edited the image written to the SLM will be updated,

and the image preview will be refreshed. By default, the center of the pattern will be placed at the center of the SLM. For some patterns it is easier to shift the “center” of the image applied to the SLM with pixel-by-pixel control rather than to optically shift the center of the illumination. Thus, the user can define the center of the image along the x and y axis.

Selecting the spiral checkbox will generate a spiral phase where Vortex Charge determines the number of waves of rotation in the image. Selecting a fork will superimpose a repeating phase ramp with the spiral phase, which simply shifts the vortex off axis.

Selecting Axicon will generate a linear phase ramp that is either increasing or decreasing from the center of the SLM image to the edge of the SLM. Setting the number of waves will determine the slope of the phase ramp.

Selecting Rings will generate a repeating concentric ring pattern. The user can specify the diameter in pixels of the inner ring, and the diameter in pixels of the outer ring. The pattern will consist of two graylevels defined by Value One and Value Two. In general, if using a 0 - 2π LUT, the two graylevels used will be 0 and 128 which will generate a 0 and π ring pattern. The ring pattern can be superimposed with defocus by setting the number of waves of defocus to apply.

After editing the image parameters, the bitmap can be saved. The default location for saving is the current path to the current sequence list.

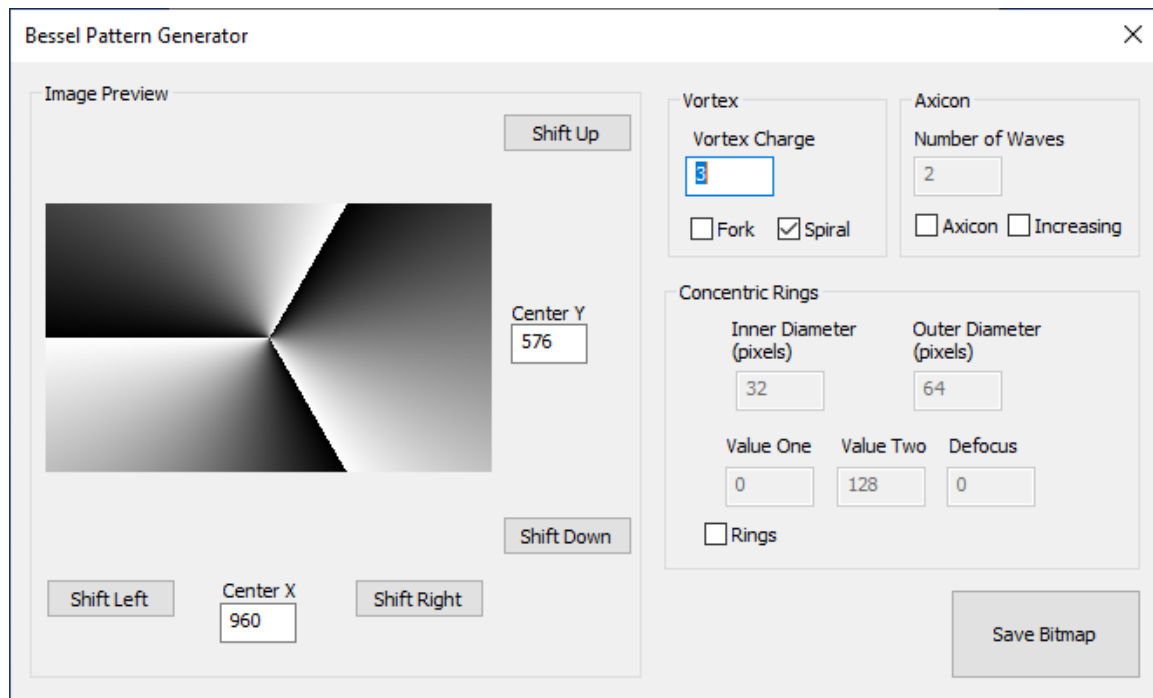


Figure 12 Bessel GUI

3.7.2 BEST Bessel Beam

The BEST Bessel Beam dialog is based on Na Ji’s Bessel Scanning Technology which enables video rate volumetric imaging in two-photon microscopes. For more information on this work, it is

highly recommended that the following paper and corresponding supplemental information be read:

Lu, Rongwen, Wenzhi Sun, Yajie Liang, Aaron Kerlin, Jens Bierfeld, Johannes D. Seelig, Daniel E. Wilson et al. "Video-rate volumetric functional imaging of the brain at synaptic resolution." *Nature neuroscience* 20, no. 4 (2017): 620.

To reduce the hurdles of software development when integrating volumetric imaging into a custom system or two photon microscope, Meadowlark Optics included a GUI (Figure 14) used to compute the theoretical axial point spread function (PSF) of a Bessel Beam based on the user's optical setup and the desired numerical aperture of excitation. To begin, the software requires information about the optical system, which is sketched in Figure 13. The user must define the wavelength of excitation, the beam diameter incident on the SLM, and the focal length of lens 1 which is placed between the SLM and the amplitude mask. Given the users desired effective numerical aperture for excitation (see Supplemental Information, Rongwen et. al.) the GUI will automatically calculate the intensity profile of the excitation beam at the amplitude mask. As outlined by Rongwen et. al., the amplitude mask can be designed to block all light except the 1st order peak and neighboring side lobes (Case I), or the amplitude mask can be designed to block all light except the 1st order peak (Case II). The specifications of the amplitude mask inner and outer radius requirements to support Case I or Case II are shown on the GUI and are dynamically updated as the parameters in the GUI are adjusted.

To calculate the axial PSF of the excitation, it is necessary to know the manufacturer, numerical aperture (NA), magnification, and immersion type (air, water, or oil) of the objective in use. Additionally, it is necessary to know the magnification of the optical relays from the amplitude mask to the galvo scanning mirror, and from the galvo scanning mirror to the objective. The optical relays should be 4F imaging systems. The magnification of the relay from the mask to the galvo is: the focal length of lens 3 divided by the focal lens 2, and the magnification of the relay from the galvo to the objective is: the focal length of lens 5 divided by the focal lens 4. If an optical relay is not found in your optical design, set the magnification in the software to 1. The axial PSF is automatically plotted as GUI parameters are adjusted, and the full width half max of the PSF is shown.

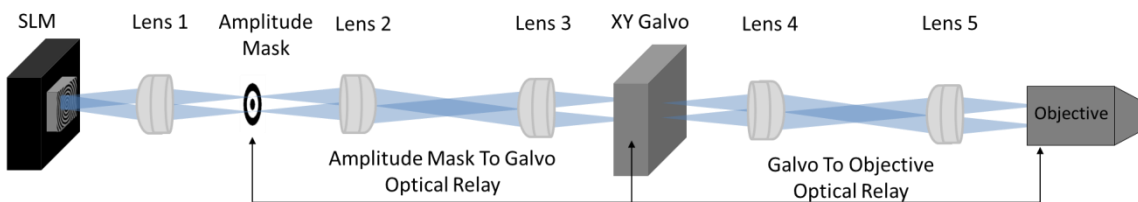


Figure 13 Optical Layout for BEST volumetric imaging

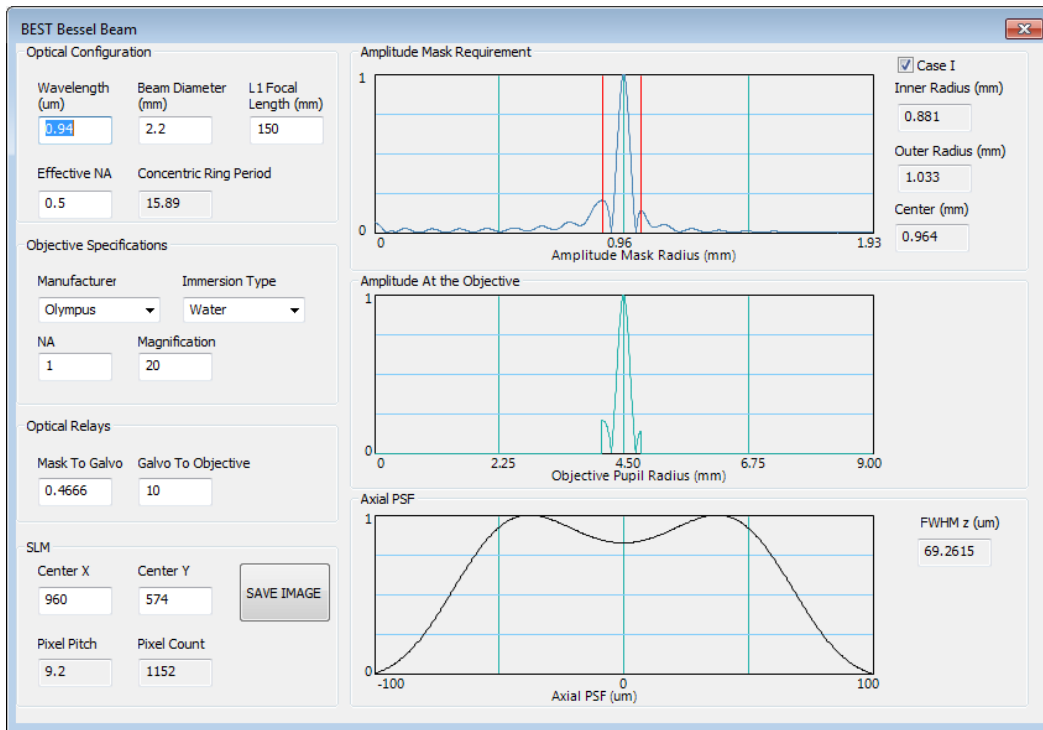


Figure 14 BEST Bessel GUI

3.7.3 Blazed or Sinusoid Grating

The blazed or sinusoidal grating GUI enables the user to create repeating ramp patterns, and sinusoidal gratings. The user must define the period in pixels from 0 - 2π , set either a horizontal or vertical grating direction, and set the slope of the grating to either increase from 0 to 2π or decrease from 2π to 0.

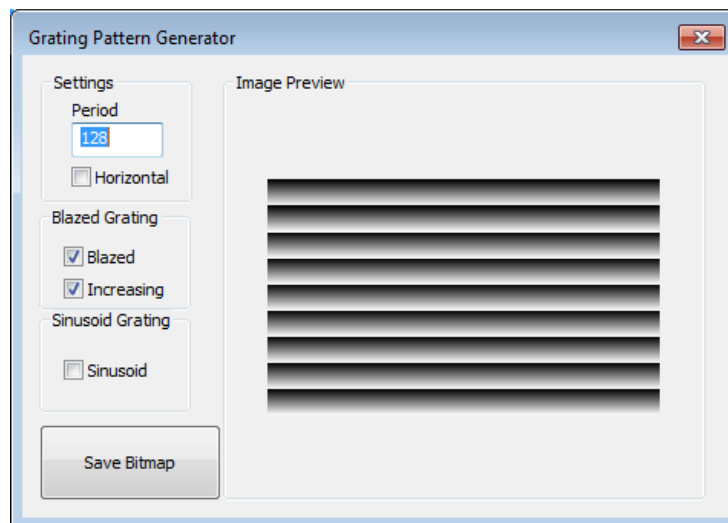


Figure 15 Grating GUI

3.7.4 Fresnel lens

The Fresnel lens GUI allows users to define either circular or cylindrical lenses. The user must define the radius SLM lens function, and the number of waves of defocus to apply over that radius. The default center of the lens is half of the SLM height and width but can be shifted to accommodate slight offsets in alignment.

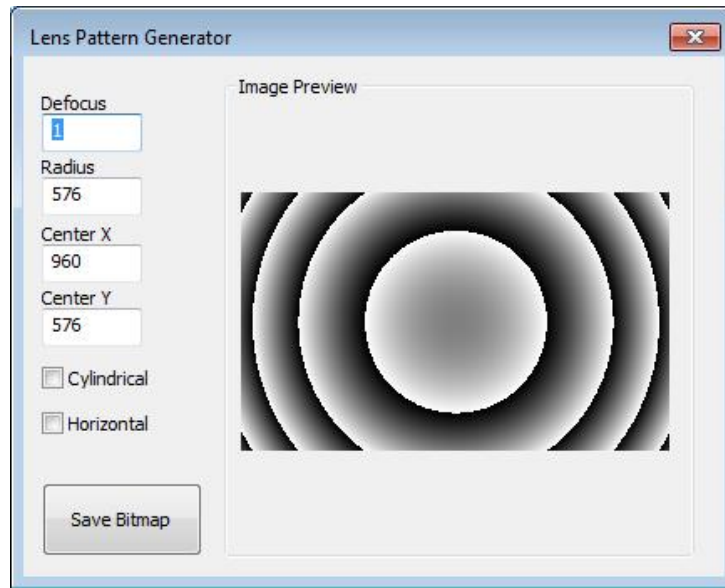


Figure 16 Fresnel Lens GUI

3.7.5 Hologram

The hologram GUI allows the user to create a phase profile that will generate a volume of focal points in the Fourier Plane. The software utilizes a Weighted Gerchberg Saxton implemented on the GPU. This is an FFT based iterative algorithm that by default allows 10 iterations to compute a desired hologram. The user can point and click on the Target Locations to create a focal point, then manually edit the x, y, z location and intensity of the focal point. Spot locations are shown in the list box in the upper left corner. Using the arrow keys on the keyboard different spot locations can be selected for editing or removal. As the hologram is computed, a preview of the hologram is shown on the GUI, and the hologram is loaded to the SLM.

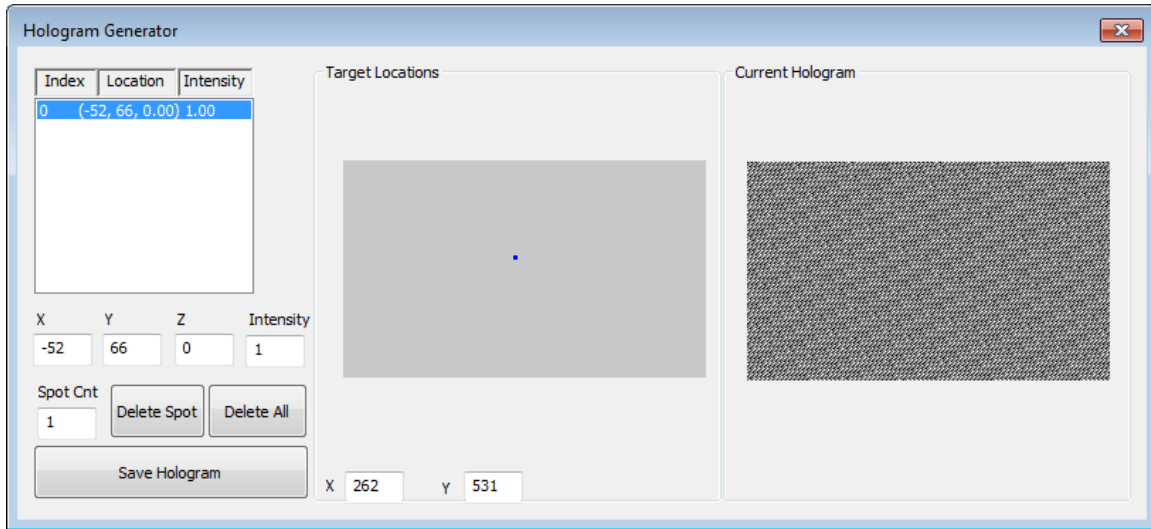


Figure 17 Hologram Generation GUI

3.7.6 Solid, Stripe, Check, or Random Phase

This GUI allows the user to create simple stripe, checkerboard, solid, or random phase patterns.

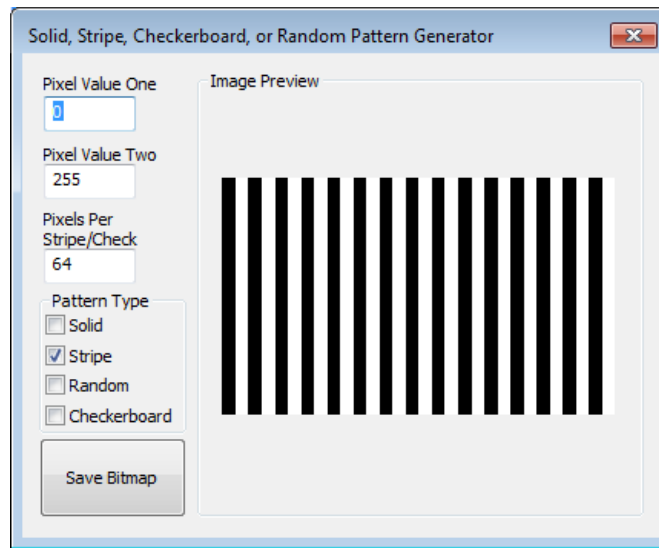


Figure 18 Solid, Stripe, Random Phase, Checkerboard GUI

3.7.7 Superimpose Images

This GUI allows the user to browse for any two images and superimpose them by adding the two images modulo 256. The output image is then processed through the LUT calibration.

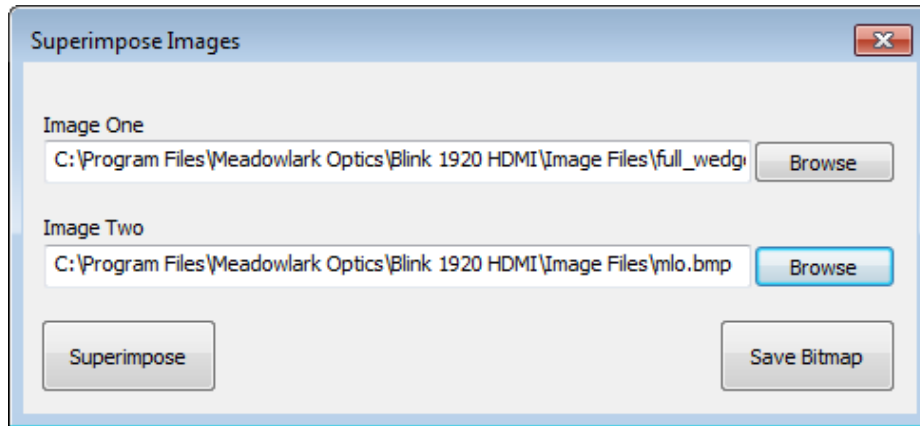


Figure 19 Superimpose Images GUI

3.7.8 Zernike Polynomials

This GUI allows the user to input Zernike polynomial coefficients and output a bitmap. The user must define a radius for the equations, the x and y location of the center of the pattern. If Zernike polynomials are being used to compute a custom aberration correction, it is recommended that the radius be a mid-point between the distance from the center to short axis edge which tends to under-correct the corners, and the center to the diagonal which tends to over-correct the edges.

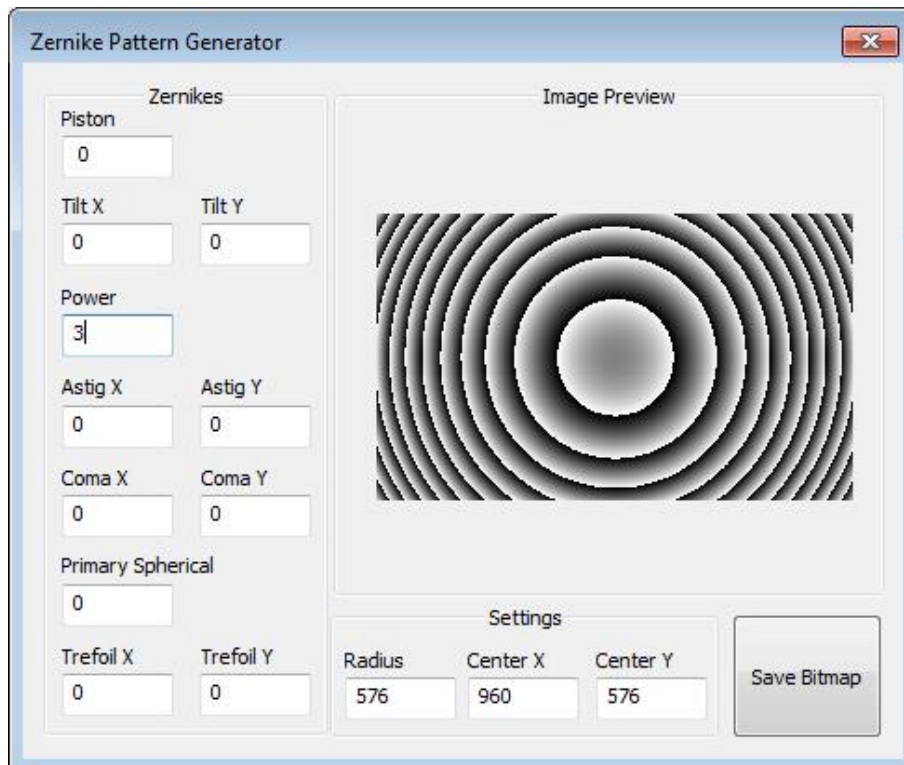


Figure 20 Zernike Polynomial GUI

4 Software Development Kit Introduction

Meadowlark Optics offers several software development kit example programs demonstrating how to interface to our SLM from Matlab, LabVIEW, C++, and Python. Each example program demonstrates the order of operations that functions should be called in, the core functions that should be used, and how to link to our Dynamic Linked Library (DLL) to drive the Spatial Light Modulator (SLM). The purpose of this document is to outline the functions available, explain the purpose of each parameter passed, and suggest appropriate values to pass for various product options. The Blink_C_wrapper DLL, and corresponding Blink_C_wrapper.h file must be used for interfacing to the SLM with Matlab, LabVIEW, Python, and C++. This document assumes familiarity with LabVIEW, Matlab, Python, and C++ in addition to the Meadowlark PCIe SLM hardware.

4.1 Modes of Operation

Figure 21 shows the order of function calls with green indicating initialization, blue indicating the customers loop of loading images to the SLM, and red indicating the proper shut down procedure. The function calls are further detailed in Function Summary and Usage.

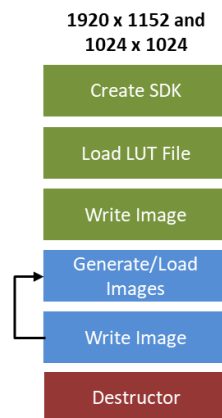


Figure 21: Function Call Order of Operations

4.2 Function Summary and Usage

From LabVIEW functions are accessed through Call Library Function Nodes. By double clicking on the function nodes LabVIEW allows the user to edit the function call and parameters, as shown in Figure 22.

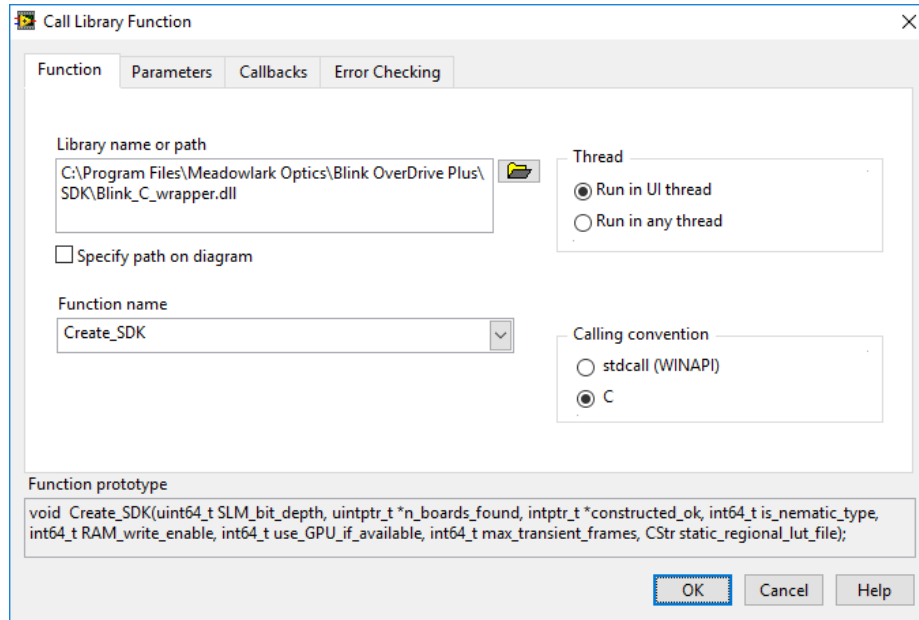


Figure 22: LabVIEW Call Library Function Node to interface to the Meadowlark Optics DLL

From Matlab the DLL is accessed by through the loadlibrary function, and subsequent calls to calllib as shown to follow. Loadlibrary takes two parameters: the Blink_C_wrapper.dll, and the .h file. The calllib function takes 'Blink_C_wrapper' as the first parameter, the function name as the second parameter, and following that the function parameters as defined in the Blink_C_wrapper.h file. **Note that matlab requires a C compiler.** We recommend installing Microsoft SDK 7.1, then at the matlab command prompt run mex – setup to select the installed compiler.

```
if ~libisloaded('Blink_C_wrapper')
    loadlibrary('Blink_C_wrapper.dll', 'Blink_C_wrapper.h');
end
```

```
sdk = calllib('Blink_SDK_C', 'Create_SDK', bit_depth, num_boards_found, constructed_okay,
is_nematic_type, RAM_write_enable, use_GPU, max_transients, lut_file);
```

4.3 Function Definitions

All function definitions listed below are based on the Blink_C_wrapper.h file that Matlab, LabVIEW, C++ and Python can interface to.

4.3.1 void Create_SDK(unsigned int* n_boards_found, int *constructed_ok);

Create SDK opens communication and initializes the hardware. The purpose, and appropriate values of the parameters passed to this function are outlined to follow.

- Number of Boards Found – In the constructor the hardware scans the PCIe bus for Meadowlark Optics Hardware. This parameter returns the number of boards found.

- Constructed OK – This parameter tells the user if the constructor completed successfully. If Constructed OK is false, it means that hardware wasn't found but the software will let the user run in simulation mode.

4.3.2 `int Load_LUT_file(int board, char* LUT_file);`

This function is used to load a calibration to the hardware that corrects for the nonlinear response of the liquid crystal to voltage. If the Look-Up Table (LUT) is correct, then the user can assume that linear increments in graylevel in user defined images translate to linear increments in phase delay on the SLM. Because images are processed through the LUT in hardware, it is important that the LUT file be loaded to the hardware prior to writing images to the SLM. The function takes a board number, which should be 1 for single SLM operation and a path to the LUT file.

Use this function to load the custom LUT file that shipped with the SLM to the hardware. In the example programs linear.lut is used as a default. The user should replace linear.LUT with the custom LUT delivered with their SLM.

The function will return 1 for success, or 0 for failure.

4.3.3 `int SetWaitForTrigger(int board, bool waitForTrigger);`

This enables external triggers to control when images are loaded to the SLM. The trigger should be a TTL pulse, applied to Input A. The hardware responds to the falling edge of the trigger. If external triggers are enabled, then the function will not return until the trigger was received. If the user attempts to trigger images before an image write is complete, then the trigger will be ignored.

The function will return 1 for success, or 0 for failure.

4.3.4 `int SetFlipImmediate(int board, bool flipImmediate);`

On the 1024 x 1024 pixel SLM it is possible to interrupt an image refresh to load a new image without completing a load of the previous image. This improves repeatability of timing but does not ensure DC balancing. **This function should be enabled with caution.**

The function will return 1 for success, or 0 for failure.

4.3.5 `int SetOutputPulse(int board, bool outputPulse);`

If enabled, this provides feedback to the user notifying when images are loaded to the SLM. If using external triggers this provides acknowledgement that the external trigger was received, and a new image was loaded to the SLM. This signal is normally high, and falls low for approximately 200 μ s.

The function will return 1 for success, or 0 for failure.

4.3.6 `int Write_image(int board, unsigned char* image, unsigned int trigger_timeout_ms);`

This function writes an image to the SLM. The function returns when the DMA is complete, not when the hardware memory bank is ready to receive a new image. To

ensure that images are never dropped you should call ImageWriteComplete after calling write image to ensure that you don't start your next DMA until the hardware is ready to receive it. The reason the write function was broken into two steps is for work with external triggers. After the DMA is complete it is safe to supply a trigger, and after ImageWriteComplete returns it is safe to start a new DMA.

Write_image function takes several parameters:

- Board – This parameter tells the software which SLM the image should be written to. For single SLM operation this should be 1.
- Image – This is a pointer to a one dimensional array containing the image data. For the 1920 x 1152 pixel SLM there should be 1920*1152, or 2,211,840 elements in the array, where each element is an 8-bit entry. For the 1024 x 1024 pixel SLM there should be 1024*1024, or 1,048,576 elements in the array, where each element is an 8-bit entry.
- Trigger Timeout – If external triggers are enabled, but they are never received by the hardware, then the software will stop waiting for a trigger when the timeout condition is met. The units of this parameter are milliseconds. In general we recommend passing 5000 for a 5 second timeout. If the timeout condition occurs the software will post an error message notifying the user of the error.

The function will return 1 for success, or 0 for failure.

4.3.7 `int ImageWriteComplete(int board, unsigned int trigger_timeout_ms);`

This function checks the hardware memory bank to verify that the hardware is ready to receive a new image. Calling this function after a DMA is initiated will ensure that images are never dropped, and the SLM will never display a partial image. On the 1920 x 1152 SLM, if external triggers are disabled, the hardware will be ready to receive an image 1.18 ms after a DMA is complete. If external triggers are enabled, the hardware will be ready to receive an image 1.18 ms after receipt of a trigger. On the 1024 x 1024 SLM, if external triggers are disabled, the hardware will be ready to receive an image 0.696 ms after a DMA is complete. If external triggers are enabled, the hardware will be ready to receive an image 0.696 ms after receipt of a trigger. It is safe to supply a trigger after Write_image returns. Triggers received before a DMA is started but after ImageWriteComplete has returned will be ignored.

ImageWriteComplete takes the following parameters:

- Board – This parameter tells the software which SLM the image should be written to. For single SLM operation this should be 1.
- Trigger Timeout – If external triggers are enabled, but they are never received by the hardware, then the software will stop waiting for a trigger when the timeout condition is met. The units of this parameter are milliseconds. In general, we recommend passing 5000 for a 5 second timeout. If the timeout condition occurs the software will post an error message notifying the user of the error.

The function will return 1 for success, or 0 for failure.

4.3.8 `int PreLoad_sequence(int board, unsigned char* image_array, int ListLength, unsigned int trigger_timeout_ms);`

This function is only used for the 1024 x 1024 pixel SLM. Pre Load Sequence loads a series of images sequentially in hardware memory. The function takes several parameters:

- Board – This parameter tells the software which SLM the image should be written to. For single SLM operation this should be 1.
- Image – This is a pointer to a one dimensional array containing the image data for “n” images. Each image contains 1024*1024 8-bit elements. The hardware can store up to 752 images. Each image in the Image array will be loaded sequentially into frames (i.e. the first image is stored in frame 0).
- List Length – the number of images stored in the Image array.
- Trigger Timeout – If external triggers are enabled, but they are never received by the hardware, then the software will stop waiting for a trigger when the timeout condition is met. The units of this parameter are milliseconds. In general we recommend passing 5000 for a 5 second timeout. If the timeout condition occurs the software will post an error message notifying the user of the error.

The function will return 1 for success, or 0 for failure.

4.3.9 `int PreLoad_Image(int board, unsigned char* image_array, int frame, unsigned int trigger_timeout_ms);`

This function is only used for the 1024 x 1024 pixel SLM. Pre Load Image loads a single image to a single frame in hardware memory. This function is useful if a customer needs to replace a single image without reloading a complete sequence, or if the customer prefers to iteratively call Pre Load Image as opposed to Pre Load Sequence. The function takes several parameters:

- Board – This parameter tells the software which SLM the image should be written to. For single SLM operation this should be 1.
- Image – This is a pointer to a one dimensional array containing the image data for “n” images. Each image contains 1024*1024 8-bit elements. The hardware can store up to 752 images. Each image in the Image array will be loaded sequentially into frames (i.e. the first image is stored in frame 0).
- Frame – which frame in memory the image should be written to. This can be a value between 0 and 752
- Trigger Timeout – If external triggers are enabled, but they are never received by the hardware, then the software will stop waiting for a trigger when the timeout condition is met. The units of this parameter are milliseconds. In general we recommend passing 5000 for a 5 second timeout. If the timeout condition occurs the software will post an error message notifying the user of the error.

The function will return 1 for success, or 0 for failure.

4.3.10 `int Select_image(int board, int frame, unsigned int flip_timeout_ms);`

This function is only used for the 1024 x 1024 pixel SLM. Select Image will send a single frame in memory out to the SLM. This function takes several parameters:

- Board – This parameter tells the software which SLM the image should be written to. For single SLM operation this should be 1.
- Frame – The hardware can store up to 752 images, this field indicates which frame in the hardware memory the image is stored at.
- Image Size – This is 1024*1024.
- Trigger Timeout – If external triggers are enabled, but they are never received by the hardware, then the software will stop waiting for a trigger when the timeout condition is met. The units of this parameter are milliseconds. In general we recommend passing 5000 for a 5 second timeout. If the timeout condition occurs the software will post an error message notifying the user of the error.

The function will return 1 for success, or 0 for failure.

4.3.11 `int StartAutoIncrement(int board, int ListLength);`

This function is only used for the 1024 x 1024 pixel SLM. If the customer has firmware rev 2.4 or higher, then the hardware supports auto increments through external triggers. This means that the customer can pre-load the hardware memory with images either by using PreLoad_Image or PreLoad_Sequence. Then the customer can call StartAutoIncrement to enable listening to external triggers. When a trigger is received the hardware will automatically load the next image in the sequence without requiring software input. This function takes several parameters:

- Board – This parameter tells the software which SLM the image should be written to. For single SLM operation this should be 1.
- List Length – The hardware can store up to 752 images, this field indicates how many frames in hardware memory are pre-loaded with the desired images. The software will automatically increment though the length of the sequence and loop until auto increment is disabled.

The function will return 1 for success, or 0 for failure.

4.3.12 `int StopAutoIncrement(int board);`

This function is only used for the 1024 x 1024 pixel SLM. This function will stop a sequence from auto-incrementing through a preloaded sequence of images. The function takes the following parameter:

- Board – This parameter tells the software which SLM the image should be written to. For single SLM operation this should be 1.

The function will return 1 for success, or 0 for failure.

4.3.13 **void Delete_SDK()**

This function destructs the handle to the hardware, properly releases memory allocations, and shuts down the hardware. This is the last function that should be called when exiting your software.

4.4 Optional Functions

4.4.1 **const char* Get_Last_error_message();**

This function is optional, but is a useful check of error messages.

4.4.2 **const char* Get_version_info();**

This function is optional, providing version information if requested by Meadowlark Optics.

4.4.3 **double Read_SLM_temperature(int board);**

This function is available with the 1920x1152, and 1024 x 1024 SLM. The function returns the temperature of the SLM backplane in degrees C.

4.4.4 **int Get_image_height(int board);**

This function returns the height of the SLM.

4.4.5 **int Get_image_width(int board);**

This function returns the width of the SLM.

4.4.6 **int Get_image_depth(int board);**

This function returns the bit depth of the SLM in bits per pixel.

4.4.7 **int SetRampDelay(int board, unsigned int ramp_delay);**

This function adjusts hardware timing on the 1920 x 1152 SLM only. We recommend that ramp delay be left at 10. The function will return 1 for success, or 0 for failure.

4.4.8 **int SetPreRampSlope(int board, unsigned int preRampSlope);**

This function adjusts hardware timing on the 1920 x 1152 SLM only, it is recommend that preRampSlope be set to 14 for amplitude modulation and 7 for phase modulation. The function will return 1 for success, or 0 for failure.

4.4.9 **int SetPostRampSlope(int board, unsigned int postRampSlope);**

This function adjusts hardware timing on the 1920 x 1152 SLM only, it is recommend that postRampSlope be set to 24. The function will return 1 for success, or 0 for failure.

4.5 Optional Image Generation Functions

Image generation functions are accessed by including ImageGen.h and linking to ImageGen.dll or ImageGen.lib.

IMPORTANT NOTE: The code used to generate holograms and apply regional LUTs requires OpenCL to be installed on your computer, and requires a graphics card that supports OpenCL. PLEASE CHECK YOUR GRAPHICS CARD CAPABILITIES.

We recommend NVIDIA graphics cards, and use of OpenCL 1.2 or higher. To check your OpenCL version you currently have installed you can download and run GPU Caps Viewer. If you are having trouble finding this program, please contact slmsupport@meadowlark.com and we will send you a link.

4.5.1 void Generate_Stripe(unsigned char* Array, unsigned char* WFC, int width, int height, int depth, int PixelValOne, int PixelValTwo, int PixelsPerStripe, int bVert, int RGB);

This function will fill an array to define an image of user defined dimensions and bit depth (8 or 10) with a stripe pattern consisting of two pixel values, and a user specified number of pixels per stripe.

The image will be automatically superimposed with the wavefront correction (WFC). The WFC and Array parameters must be pre-allocated with either: 1920*1152 unsigned 8-bit elements, or 1024*1024 unsigned 8-bit elements depending on the SLM in use. Passing 0 or 1 to bVert will change the direction of the grating between vertical and horizontal. The RGB parameter is always false for a PCIe interface.

4.5.2 void Generate_Checkerboard(unsigned char* Array, unsigned char* WFC, int width, int height, int depth, int PixelValOne, int PixelValTwo, int PixelsPerCheck, int RGB);

This function will fill an array to define an image of user defined dimensions and bit depth (8 or 10) with a checkerboard pattern consisting of two pixel values, and a user specified number of pixels per stripe.

The image will be automatically superimposed with the wavefront correction (WFC). The WFC and Array parameters must be pre-allocated with either: 1920*1152 unsigned 8-bit elements, or 1024*1024 unsigned 8-bit elements depending on the SLM in use. The RGB parameter is always false for a PCIe interface.

4.5.3 void Generate_Solid(unsigned char* Array, unsigned char* WFC, int width, int height, int depth, int PixelVal, int RGB);

This function will fill an array to define an image of user defined dimensions and bit depth (8 or 10) with a solid graylevel.

The image will be automatically superimposed with the wavefront correction (WFC). The WFC and Array parameters must be pre-allocated with either: 1920*1152 unsigned 8-bit elements, or 1024*1024 unsigned 8-bit elements depending on the SLM in use. The RGB parameter is always false for a PCIe interface.

4.5.4 void Generate_Random(unsigned char* Array, unsigned char* WFC, int width, int height, int depth, int RGB);

This function will fill an array to define an image of user defined dimensions and bit depth (8 or 10) with a random phase pattern.

The image will be automatically superimposed with the wavefront correction (WFC). The WFC and Array parameters must be pre-allocated with either: 1920*1152 unsigned 8-bit elements, or 1024*1024 unsigned 8-bit elements depending on the SLM in use. The RGB parameter is always false for a PCIe interface.

4.5.5 void Generate_Zernike(unsigned char* Array unsigned char* WFC, int width, int height, int depth, int CenterX, int CenterY, int Radius, float Piston, float TiltX, float TiltY, float Power, float AstigX, float AstigY, float ComaX, float ComaY, float PrimarySpherical, float TrefoilX, float TrefoilY, float SecondaryAstigX, float SecondaryAstigY, float SecondaryComaX, float SecondaryComaY, float SecondarySpherical, float TetrafoilX, float TetrafoilY, float TertiarySpherical, float QuaternarySpherical, int RGB);

This function will fill an array to define an image of user defined dimensions and bit depth (8 or 10) using Zernike polynomials. The Zernikes center is defined by center x and center y. The radius defines the number of pixels over which one wave of phase change should occur.

The image will be automatically superimposed with the wavefront correction (WFC). The WFC and Array parameters must be pre-allocated with either: 1920*1152 unsigned 8-bit elements, or 1024*1024 unsigned 8-bit elements depending on the SLM in use. The RGB parameter is always false for a PCIe interface.

4.5.6 void Generate_FresnelLens(unsigned char* Array, unsigned char* WFC, int width, int height, int depth, int CenterX, int CenterY, int Radius, double power, bool cylindrical, int horizontal, int RGB);

This function will fill an array to define an image of user defined dimensions and bit depth (8 or 10) with a lens function. The center of the lens is defined by center x and center y. The radius defines the number of pixels over which one wave of phase change should occur. Setting cylindrical will either generate a circular or cylindrical lens, if cylindrical the lens can be horizontal or vertical.

The image will be automatically superimposed with the wavefront correction (WFC). The WFC and Array parameters must be pre-allocated with either: 1920*1152 unsigned 8-bit elements, or

1024*1024 unsigned 8-bit elements depending on the SLM in use. The RGB parameter is always false for a PCIe interface.

4.5.7 void Generate_Grating(unsigned char* Array, unsigned char* WFC, int width, int height, int depth, int Period, bool increasing, int horizontal, int RGB);

This function will fill an array to define an image of user defined dimensions and bit depth (8 or 10) with a repeating phase ramp. The user must specify the period of the ramp, if the ramp is increasing or decreasing, and if the grating is horizontal or vertical.

The image will be automatically superimposed with the wavefront correction (WFC). The WFC and Array parameters must be pre-allocated with either: 1920*1152 unsigned 8-bit elements, or 1024*1024 unsigned 8-bit elements depending on the SLM in use. The RGB parameter is always false for a PCIe interface.

4.5.8 void Generate_Sinusoid(unsigned char* Array, unsigned char* WFC, int width, int height, int depth, int Period, int horizontal, int RGB);

This function will fill an array to define an image of user defined dimensions and bit depth (8 or 10) with a repeating phase ramp. The user must specify the period of the ramp, and if the grating is horizontal or vertical.

The image will be automatically superimposed with the wavefront correction (WFC). The WFC and Array parameters must be pre-allocated with either: 1920*1152 unsigned 8-bit elements, or 1024*1024 unsigned 8-bit elements depending on the SLM in use. The RGB parameter is always false for a PCIe interface.

4.5.9 void Generate_LG(unsigned char* Array, unsigned char* WFC, int width, int height, int depth, int VortexCharge, int centerX, int center, int fork, int RGB);

This function will fill an array to define an image of user defined dimensions and bit depth (8 or 10) with a spiral phase. The user should specify the vortex charge (the number of waves of phase delay in the spiral), the x and y location of the center discontinuity, and if a tilt function should be superimposed with the grating.

The image will be automatically superimposed with the wavefront correction (WFC). The WFC and Array parameters must be pre-allocated with either: 1920*1152 unsigned 8-bit elements, or 1024*1024 unsigned 8-bit elements depending on the SLM in use. The RGB parameter is always false for a PCIe interface.

4.5.10 void Generate_ConcentricRings(unsigned char* Array, unsigned char* WFC, int width, int height, int depth, int InnerDiameter, int OuterDiameter, int PixelValOne, int PixelValTwo, int centerX, int centerY, int RGB);

This function will fill an array to define an image of user defined dimensions and bit depth (8 or 10) with concentric rings to make a Bessel beam. The user should specify the inner and outer

diameter of the repeating ring structure, the graylevel of the two rings, and the x and y location of the center discontinuity.

The image will be automatically superimposed with the wavefront correction (WFC). The WFC and Array parameters must be pre-allocated with either: 1920*1152 unsigned 8-bit elements, or 1024*1024 unsigned 8-bit elements depending on the SLM in use. The RGB parameter is always false for a PCIe interface.

4.5.11 void Generate_Axicon(unsigned char* Array, unsigned char* WFC, int width, int height, int depth, int PhaseDelay, int centerX, int centerY, int increasing, int RGB);

This function will fill an array to define an image of user defined dimensions and bit depth (8 or 10) with a radially symmetric linear phase ramp. The user should specify the number of waves of phase delay, the x and y location of the center discontinuity, and if the phase delay is increasing or decreasing.

The image will be automatically superimposed with the wavefront correction (WFC). The WFC and Array parameters must be pre-allocated with either: 1920*1152 unsigned 8-bit elements, or 1024*1024 unsigned 8-bit elements depending on the SLM in use. The RGB parameter is always false for a PCIe interface.

4.5.12 bool Initialize_HologramGenerator(int width, int height, int depth, int iterations, int RGB);

The hologram generator relies on the GPU to complete the required computations. This function opens communication with the GPU, and specifies the height and width of the hologram that will be computed such that buffers can be allocated. This also allocates the number of iterations that will be used in computing the hologram. The RGB parameter is always false for a PCIe interface.

4.5.13 int CalculateAffinePolynomials(int SLM_X_0, int SLM_Y_0, int CAM_X_0, int CAM_Y_0, int SLM_X_1, int SLM_Y_1, int CAM_X_1, int CAM_Y_1, int SLM_X_2, int SLM_Y_2, int CAM_X_2, int CAM_Y_2);

An affine transformation can be applied to the hologram computation algorithm to apply a mapping between focal point locations in the image plane, and focal point locations generated by the SLM. To calculate the transformation the user should create a series of three focal points in XY locations in SLM coordinates and record the resulting location of the 1st order focal point at a detector in the Fourier Plane. For example, Figure 23 shows one of the three measurements that must be passed as parameters to CalculateAffinePolynomials: a focal point with a SLM XY location of 128, 128, and a corresponding first order focal point on a camera. The locations will be passed as SLM_X_0 (128), SLM_Y_0 (128), CAM_X_0 (the x location of the 1st order), and CAM_Y_0 (the y location of the 1st order). To get a good calibration it is recommended that focal point locations are recorded when steering a focal point to three different quadrants. For example, SLM XY locations of: (128,128), (-128, 128), and (-128, -128). If 128 is too close to the 0th order to get an accurate measurement a wider angle can be used, such as 256 or 512.

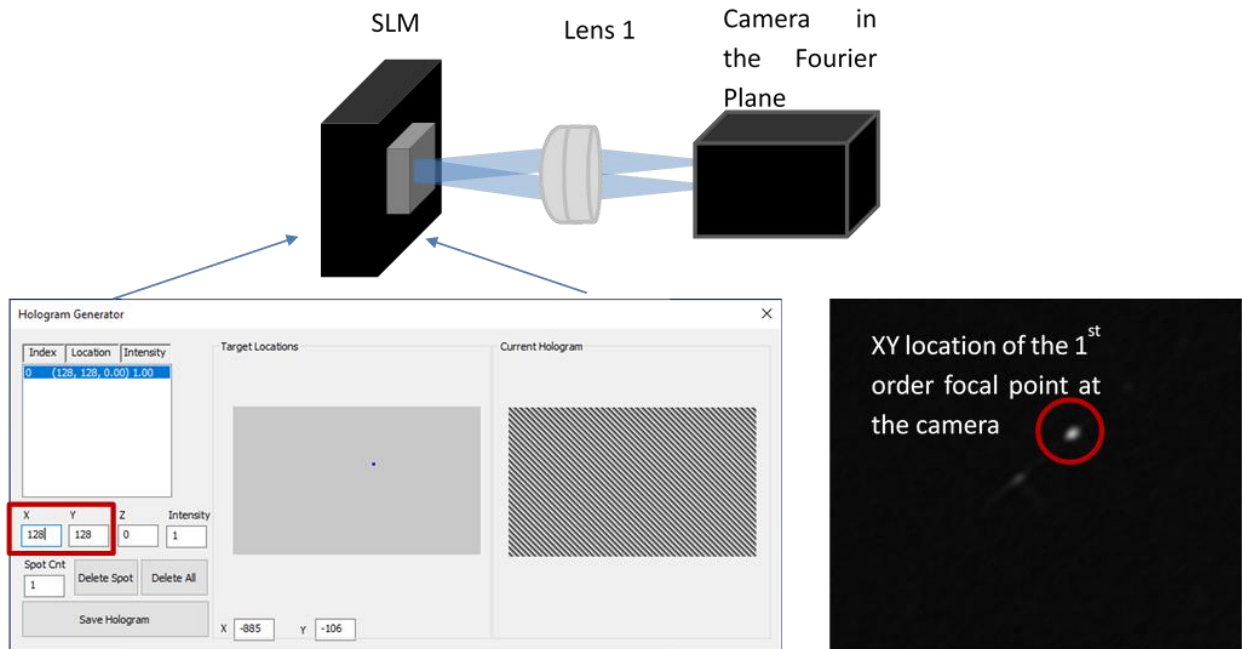


Figure 23 Generating an Affine Calibration

4.5.14 `bool Generate_Hologram(unsigned char *Array, unsigned char *WFC, float *x_spots, float *y_spots, float *z_spots, float *I_spots, int N_spots, int ApplyAffine);`

This function computes a hologram to create one or more focal points in the Fourier Plane using a Weighted Gerchberg Saxton. The user should supply an array to be filled with image data by the GPU, an array of x,y,z locations and corresponding focal point intensities, the number of focal points that the user is supplying, and indicate if the affine transformation should be applied. To apply the affine transformation, the user should have already made a call to CalculateAffinePolynomials to generate the required mapping of SLM locations to camera coordinates.

The image will be automatically superimposed with the wavefront correction (WFC). The WFC and Array parameters must be pre-allocated with either: 1920*1152 unsigned 8-bit elements, or 1024*1024 unsigned 8-bit elements depending on the SLM in use. The RGB parameter is always false for a PCIe interface.

4.5.15 `void Destruct_HologramGenerator();`

This must be called when done using the GPU to compute holograms.

4.5.16 `bool Initialize_RegionalLUT(int width, int height, int depth);`

A look up table (LUT) calibration to linearize the optical response of the liquid crystal to applied voltage. A regional LUT is generated by characterizing the phase vs. voltage response regionally (see section 6) such that the calibration is spatially correct across the SLM. This calibration relies on the GPU to take a table of discrete measurements and generate a pixel-by-pixel phase calibration to be applied to the image. This function opens communication with the GPU, and

specifies the height and width of the images that will be processed through the calibration such that buffers can be allocated.

4.5.17 *bool Load_RegionalLUT(const char* const RegionalLUTPath, float* Max, float* Min);*

Using the directions in Section 6 a regional calibration can be generated. This function loads the static table of discrete measurements to the GPU such that the pixel-by-pixel calibration map can be calculated. The function takes a path to the regional LUT file, and fills the contents of empty variables (Max and Min) with the maximum and minimum graylevels found in the regional LUT. The regional LUT is applied in software, and the global LUT applied in hardware should be defined by the user as a linear LUT that ranges from the minimum graylevel in the regional LUT to the maximum graylevel in the regional LUT.

4.5.18 *bool Apply_RegionalLUT(unsigned char *Array);*

This function processes images through the pixel-by-pixel phase calibration. This must be called prior to loading the image to the hardware.

4.5.19 *void Destruct_RegionalLUT();*

This must be called when done using the GPU to apply regional phase calibrations to images.

4.5.20 *void Mask_Image(unsigned char* Array, int width, int height, int depth, int region, int NumRegions, int RGB);*

This function is used in the calibration of a regional LUT to mask off all of an image to 0 except the specified region. For example, if NumRegions is 64, and region is 0, then the SLM is divided into 8x8 regions, and all image data outside region 0 is set to 0. The RGB parameter is always false for a PCIe interface.

4.5.21 *bool SetBESTConstants(int FocalLength, float BeamDiameter, float Wavelength, float SLMpitch, int SLMNumPixels, float ObjNA, float ObjMag, float ObjReflnd, float TubeLength, float RelayMag);*

This function sets optical constants that are used in the BEST Bessel beam calculation. Focal Length is the focal length of the lens between the SLM and the amplitude mask. Beam diameter is the diameter of the beam incident on the SLM. Wavelength is the wavelength of excitation. SLM pitch is the pixel pitch of the SLM model used. SLM Num Pixels is the number of pixels along the short axis of the SLM. The objective specifications are the NA, Magnification, and refractive index (of air, water immersion if applicable, or oil immersion if applicable). The tube length depends on the objective manufacturer: Olympus objectives should use a tube length of 180, Nikon of 200, Zeiss of 165, and Leica of 200. The relay magnification is the total magnification from the amplitude mask to the objective.

4.5.22 bool GetBESTAmplitudeMask(float* AmplitudeY, float* Peaks, int* PeaksIndex, float Period);

This function calculates the lateral intensity profile of the excitation at the amplitude mask. The function takes an empty array. The elements in the array can be computed as follows:

Objective Pupil Radius = (Objective NA*Tube Length) / objective magnification

Center Mask = (effective NA * Objective Pupil Radius) / Total Optical Relay Magnification

Amplitude Y Array Length = (Center Mask * 1000 * 2) + 1

The Peaks and Peaks index arrays are arrays with five elements that are used to note the value of the peaks and nulls along the axial intensity profile, as well as the index in the array that contains the peaks and nulls. The indices are noted as follows:

0 – Inner Peak

1 – Inner Null

2 – Center Peak

3 – Outer Null

4 – Outer Peak

If using Case I as described in Rongwen et. al., then the amplitude mask rings should cut on the inner peak and the outer peak. If using Case II, then the amplitude mask rings should cut on the inner null and the outer null. For more information please read the following paper and corresponding supplemental information.

Lu, Rongwen, Wenzhi Sun, Yajie Liang, Aaron Kerlin, Jens Bierfeld, Johannes D. Seelig, Daniel E. Wilson et al. "Video-rate volumetric functional imaging of the brain at synaptic resolution." *Nature neuroscience* 20, no. 4 (2017): 620.

The last parameter passed to this function is the period of concentric ring pattern, which is a floating point number. This is calculated as:

Period = Focal Length Lens 1 * wavelength / SLM pixel pitch / Center Mask;

4.5.23 bool GetBESTAxialPSF(float* axialAmplitude, float* Intensity, float Period, float OuterDiameter, float InnerDiameter);

This function calculates the axial PSF of the Bessel excitation beam. The first parameter is an empty array that will be filled with the axial intensity profile. The length of this array is calculated as:

Axial Amplitude Length = Objective Pupil Radius / Total Optical Relay Mag. * 1000 * 2);

The Intensity array is also an empty array that is filled with the lateral intensity of the excitation at the objective. The length of this array is the same as the Axial Amplitude Length. Last the function takes the period of the concentric ring pattern (see 5.5.21) and the diameter of the amplitude mask cutoff.

If using Case I

Outer Diameter = $2.0 * \text{Peaks}[\text{OuterPeak}]$; //mm, outer diameter of annular ring in the MASK
Inner Diameter = $2.0 * \text{Peaks}[\text{InnerPeak}]$;

If using Case II

Outer Diameter = $2.0 * \text{Peaks}[\text{OuterNull}]$; //mm, outer diameter of annular ring in the MASK
Inner Diameter = $2.0 * \text{Peaks}[\text{InnerNull}]$;

4.5.24 void Generate_BESTRings(unsigned char* Array, unsigned char* WFC, int width, int height, int depth, int centerX, int centerY, float S, int RGB);

The function takes an empty array that will be filled with concentric ring image data, the width and height of the SLM, and x and y center of the SLM image (generally half the SLM height and half the SLM width), and a floating-point period for the concentric ring period.

The image will be automatically superimposed with the wavefront correction (WFC). The WFC and Array parameters must be pre-allocated with either: 1920*1152 unsigned 8-bit elements, or 1024*1024 unsigned 8-bit elements depending on the SLM in use. The RGB parameter is always false for a PCIe interface.

5 Generating a Custom LUT Calibration

5.1 Background

All Meadowlark LCOS spatial light modulators ship with a custom calibration file called a LUT file (look-up table). The LUT files are generated with one of the standard laser wavelengths available at Meadowlark Optics (405 nm, 532 nm, 635 nm, 785 nm, 1064 nm, or 1550 nm). A LUT file is required because the optical response of liquid crystal (LC) to applied voltage is nonlinear. If your optical system uses a different wavelength, or a different angle of incidence, or a different incident power then generating a calibration in your optical system is highly recommended. This document will walk you through the background of the process we recommend for calibrating the SLM and the steps you should follow to generate your own calibration.

5.2 Introduction to the Diffractive Calibration Method

The method Meadowlark utilizes to calibrate measures the optical response of the SLM as a function of applied voltage using diffraction. By tuning the frequency of the diffraction pattern used in the calibration to match the frequency of patterns in your experiments the calibration can be optimized for your experimental conditions. For example, if utilizing high frequency holograms it is recommended that a high frequency diffraction pattern be used to generate the calibration. The provided post processing software supports several modes of operation. Either the user can browse to a set of measurements you collected and map input gray levels to output phase, or measurements can be automatically collected and post processed dynamically. The output of the post processing software is either a regional or a global LUT file that is used to linearize the phase response between 0 and $n\pi$ (in most cases 0 and 2π).

Figure 24 shows a typical optical setup for diffractive calibration. It is ideal if the illumination angle of incidence during calibrations matches that which will be used in experiments. If a global calibration is being generated, it is recommended the SLM should be illuminated with a beam that covers the active area of the SLM without extending beyond the active area. This will minimize losses. If a regional calibration is being generated, then it is recommended that the incident illumination be as uniform as possible without reducing the regional intensity to the point that the 1st order diffraction pattern is in the noise. A lens should be selected with a long focal length (~ 100 , or 150 mm) to provide enough separation between the 0th order and the diffraction orders to align the detector to collect the 1st order. An iris may be used in front of the detector to block detection of light from unwanted diffraction orders. The detector used can be a simple Si or InGaAs photodetector, such as the Thorlabs DET36A2. For dynamic calibrations DiffractionLUT.exe must be able to read the intensity recorded from the detector for each applied pattern to the SLM. Our software has been designed to operate with a National Instruments Data Acquisition card (USB-6008 or similar). **Note to operate our software NI-MAX drivers must be installed on your computer or the required DLLs will be missing.**

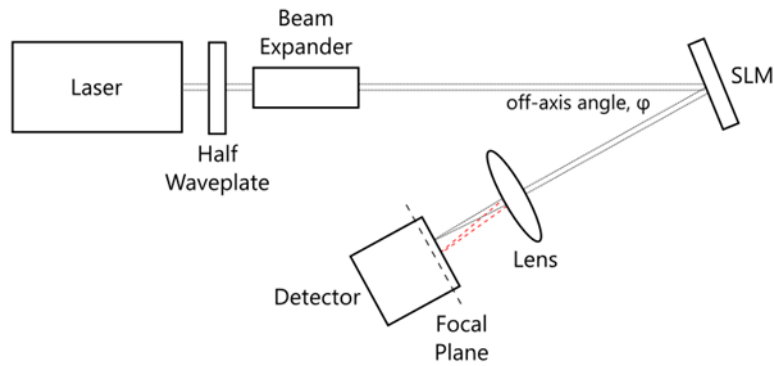


Figure 24 Typical optical setup for diffractive calibration

Figure 25 shows typical raw measurements collected utilizing the diffractive method. In this case the 1st order was measured as the diffraction patterns were loaded to the SLM. Collection the 0th or 1st order is acceptable for a global calibration, but a regional calibration requires collection of the 1st order. Delta Grayscale indicates the difference in grayscale between the reference and variable stripe. The dotted lines represent the phase shift locations for π and 2π . The first null or peak (depending on if the 0th order, or 1st order is collected) is π and the second null or peak is 2π .

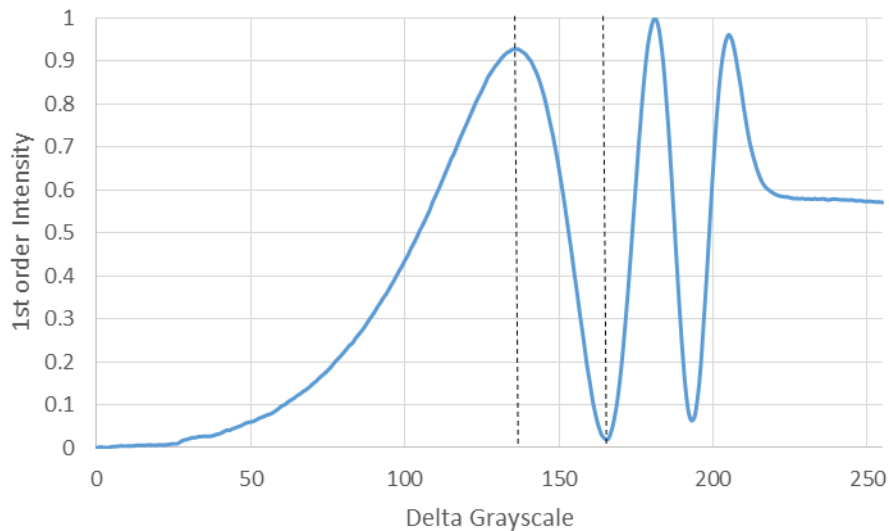


Figure 25 Typical diffractive calibration data

Figure 26 shows a simulated Fourier plane generated by the SLM when the diffractive calibration method is used. If writing binary phase grating stripes to the SLM a $\pm 1^{\text{st}}$ order will be generated about the 0th order. In order to generate a LUT, the customer should align the detector to collect the +1 or the 0th order.

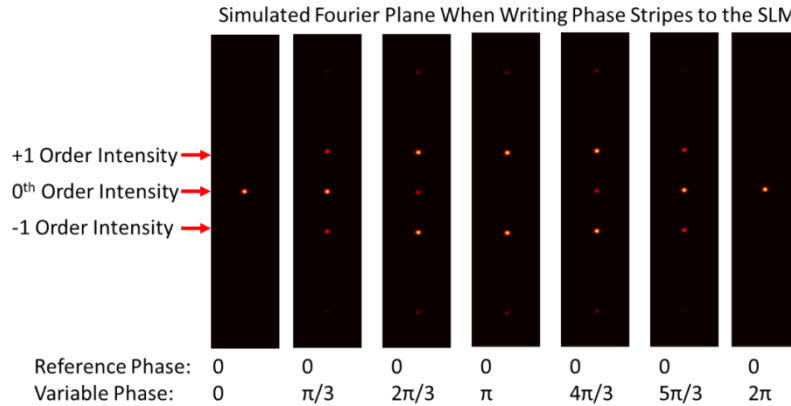


Figure 26 Simulated Fourier Plane with stripe patterns

5.3 Collecting and Post Processing Static measurements

For post processing of pre-determined static measurements, it is assumed that the user has developed a program to write diffraction gratings to the SLM, and recorded 1st order intensity. The applied diffraction gratings should be stripes with one value held constant at 0 for the 1920 x 1152, or 255 for the 1024 x 1024. The other value will vary from 0 to 255 for the 1920 x 1152, or 255 to 0 for the 1024 x 1024. The width of the diffraction grating pattern written to the SLM during this test should be sufficiently small to clearly separate the 0th and 1st order. We typically use, and recommend, 4 pixels per stripe. The customer should use a photodetector and analog input board to measure the intensity of either the zeroth or first order diffraction spot as each pattern is loaded to the SLM. The output should be saved as a *.csv file with the first column indicating the difference in grayscale between the reference and variable, and the second column should be the corresponding intensity recorded.

To facilitate this method of calibration, Meadowlark optics provides example programs written in C++, Matlab, and LabVIEW that can be used to collect the raw measurements required to generate a LUT. The example programs are named PCleDiffractiveTest (.cpp, .m, or .vi). The order of operations in the examples is as follows:

1. Open communication to the SLM.
2. Load a linear LUT to the SLM – this effectively disables the LUT such that the raw optical response of the liquid crystal to applied voltage can be measured.
3. Loop through the number of regions in the calibration. A global LUT will use one region, or a regional LUT will use 64 regions. The measurements of regional data are built into a pixel-by-pixel 3D map to spatially linearize the phase response of the SLM from 0 – 'n'π.
4. Loop through the 256 diffraction patterns. First, the example generates a stripe image. To maximize the SLM switching speed the calibration should be generated from the highest voltage available down to a voltage that gives a phase difference of 2π. However, in some cases customers will opt to calibrate at a lower starting voltage to minimize phase ripple. The table below summarizes the SLM model, grayscale, and corresponding pixel voltage.

	Grayscale, Voltage	Grayscale, Voltage	Maximize Switching Speed	Minimize Ripple
1920 x 1152 8-bit/12-bit	Gray 0, 5 V Pixel	Gray 255, 0 V Pixel	Reference = 0, Variable Grayscale: 0 to 255	Ref > 0, Variable Grayscale: Ref to 255
1024 x 1024 8-bit/12-bit	Gray 255, 5 V Pixel	Gray 0, 0 V Pixel	Reference = 255, Variable Grayscale: 255 to 0	Ref < 255, Variable Grayscale: Ref to 0

Next, if generating a regional calibration, the image is masked off to graylevel 0 except the current region of interest. This masked image is written to the SLM.

5. Allow a delay to let the liquid crystal to settle into the new phase pattern. We recommend 10 ms.
6. After the phase pattern is written to the SLM, **you must fill in your own code to read from your particular analog input board.**
7. After each region is measured, the raw intensity vs. grayscale measurements are stored in Raw[region].csv to be post processed by DiffractionLUT.exe. For example, if generating a global LUT there will be one file generated named **Raw0.csv**, and if generating a regional LUT there will be 64 files named **Raw0.csv** to **Raw63.csv**.
8. Post process through the Diffraction LUT by browsing for your Raw*.csv file(s) and then click the Save LUT button.

5.4 Dynamic Calibration of LUT Files

For dynamic LUT calibration the user should have the SLM attached and powered prior to opening Diffraction LUT. The software will be prepared to read from a National Instruments DAQ board. As shown in Figure 27, use National Instruments Measurement and Automation explorer to determine the Channel Name assigned to your DAQ board. The default Channel can be manually updated in C:\Program Files\Meadowlark Optics\Blink Plus\Cal Kit\Preferences.ini or directly through the Diffraction LUT GUI. Once the program is open you will see the GUI shown in Figure 28.

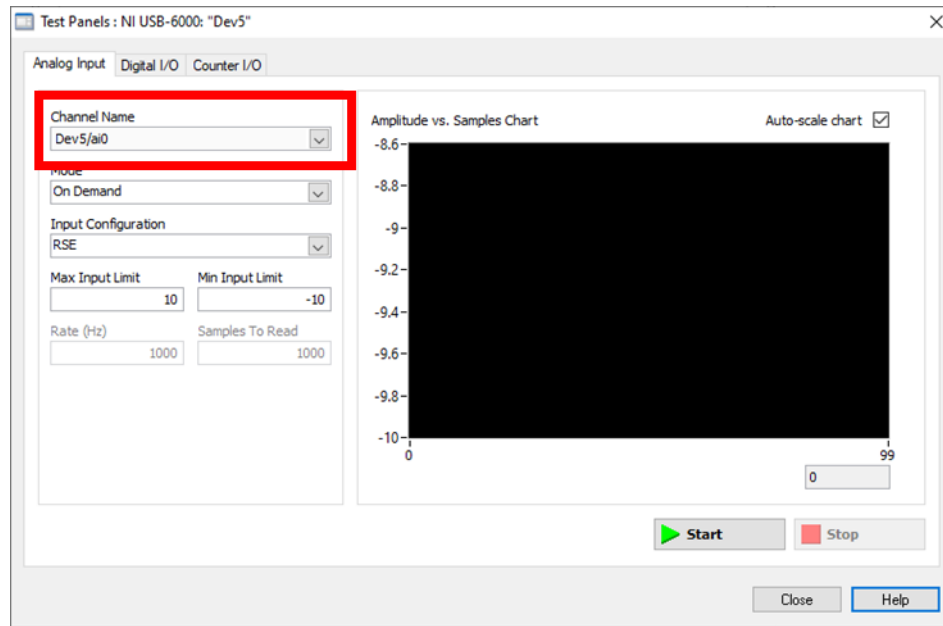


Figure 27 National Instruments Measurement and Automation Explorer used to read from your DAQ board.

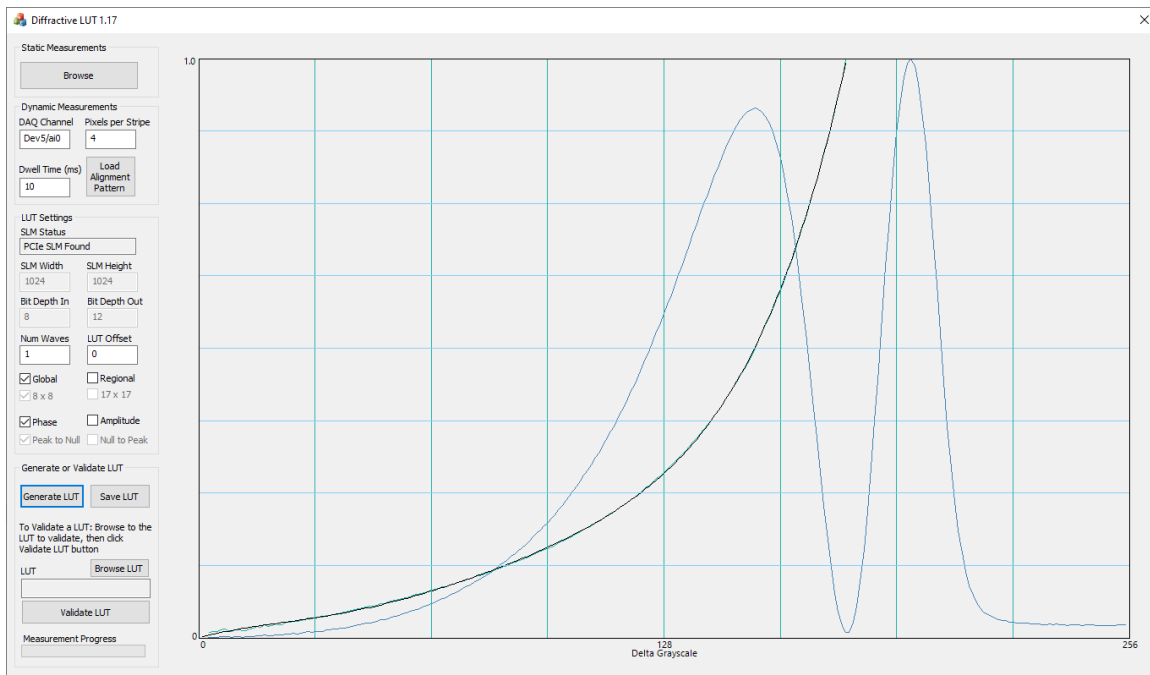


Figure 28 Diffractive LUT GUI, shown with PCIe SLM attached for dynamic LUT generation.

The GUI will automatically scan for a SLM and update the SLM height, width, input bit depth, and output bit depth. It is recommended that Pixels Per stripe be left at 4, and Dwell Time be left at 10 ms. Click the 'Load Alignment Button' to write a diffraction grating to your SLM, then use the NI test panel to adjust the position of your 1st order focal point onto your detector. Then close the NI Test panel. Click Generate LUT to collect measurements and post process. **Note that the process of loading the LUT used to calibrate, and loading images while measuring the diffracted**

Meadowlark Optics, Inc.

order takes several minutes. The GUI will show a busy mouse when loading the LUT, and then will update the measurement progress as it is loading calibration images and collecting measurements. When it is done the graph will update. After measurements have been collected, the Save LUT button will be enabled. There are several features of the GUI that customers should be aware of:

1. **Num Waves.** In most cases the user will calibrate over 1 wave ($0 - 2\pi$). However, the user can dynamically set the number of waves the calibration linearizes over: 0.5 waves = 0 to 1π , 1 wave = 0 to 2π , 1.5 waves = 0 to 3π , and 2 waves = 0 to 4π . After editing Num Waves the plot of the intensity to phase measurement and the curve fit will automatically update.
2. **LUT Offset.** The default process calibrates from a high voltage down to a phase difference of 2π . This results in the fastest response time, however it also results in the highest phase ripple. If the customer would like to reduce phase ripple the LUT offset can be used to shift the voltage range of the calibration. The voltage shift should be selected such that the SLM still has more than 1 wave of modulation. For example, in Figure 29 a LUT offset of 100 was used. This reduced the maximum voltage applied, and thus reduced the available modulation depth as compared to Figure 28. However, the SLM still has a full wave of modulation to calibrate over, and the shifted LUT can be saved.

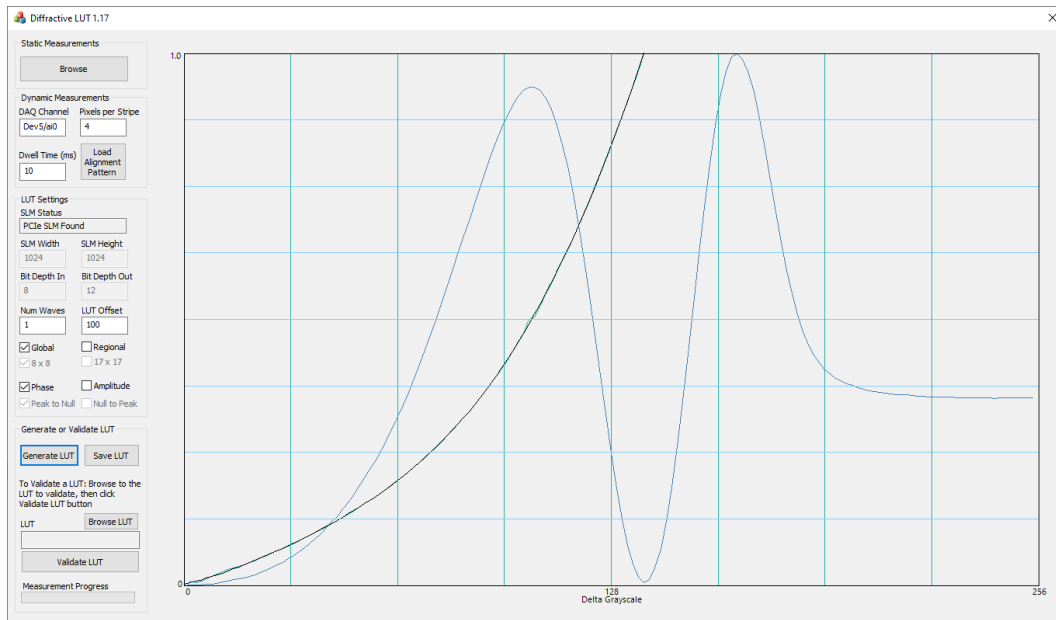


Figure 29 Use LUT Offset to shift the voltage range of the LUT to maximize phase stability.

3. **Browse LUT and Validate LUT.** To validate a LUT, browse to the LUT file in question. This will load the LUT to the SLM controller. Click the Validate LUT button to measure the optical response with the LUT applied. If the LUT is properly calibrated the SLM will have one wave of modulation with a linear phase vs. voltage response as shown in Figure 30. Note that the 2π null will not be as low as the 0π null due to inter-pixel smoothing at the phase wraps as opposed to a theoretical perfect phase discontinuity.

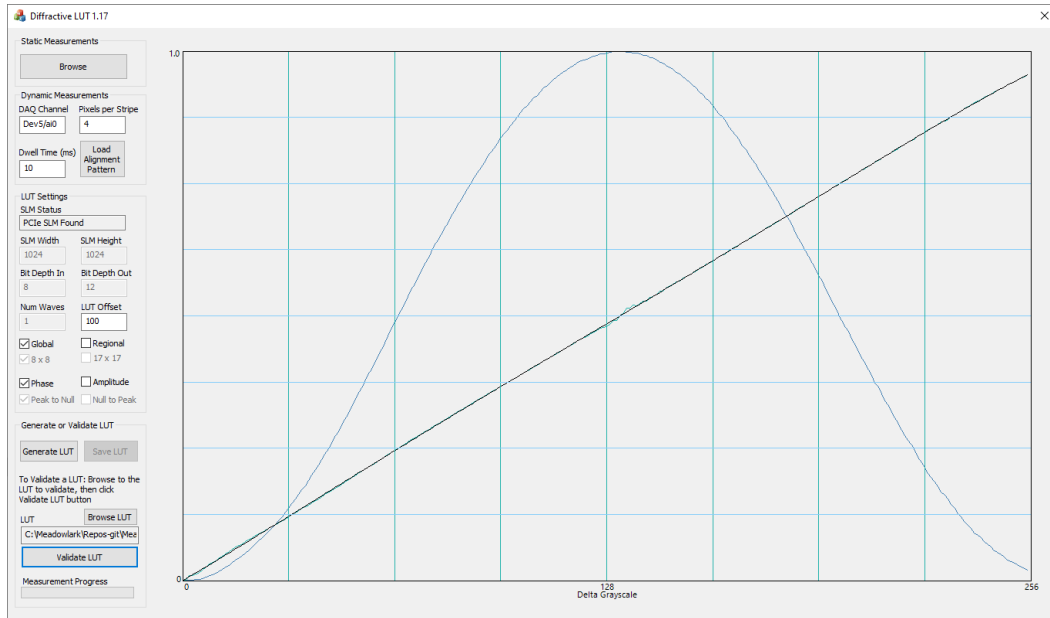


Figure 30 Validating a calibrated LUT. The phase vs. voltage should be linear from 0 to 2π

4. **Regional Calibration.** It is unusual for a regional calibration to be required. It generally results in a 1% improvement in diffraction efficiency if Gaussian illumination is utilized. This is due to strong uniformity in the center of the SLM. However, if desired, the SLM can be broken into 8 x 8 or 17 x 17 subsections to measure the optical response vs. voltage and a regional LUT can be generated. Note that the regional LUT is applied in software, so using the regional LUT adds additional latencies.
5. **Amplitude Calibration.** For an amplitude calibration indicate if the LUT should be generated from a null to a peak, or a peak to a null.

5 Mechanical Housing

The SLM Housing has vertical and horizontal micrometers for adjusting the tip and tilt up to $\pm 3^\circ$. The housing also allows the user to rotate the SLM about the optical axis by $\sim 15^\circ$. A set screw (located between the tip/tilt knobs on the side of the SLM housing) locks the rotation about the optical axis.

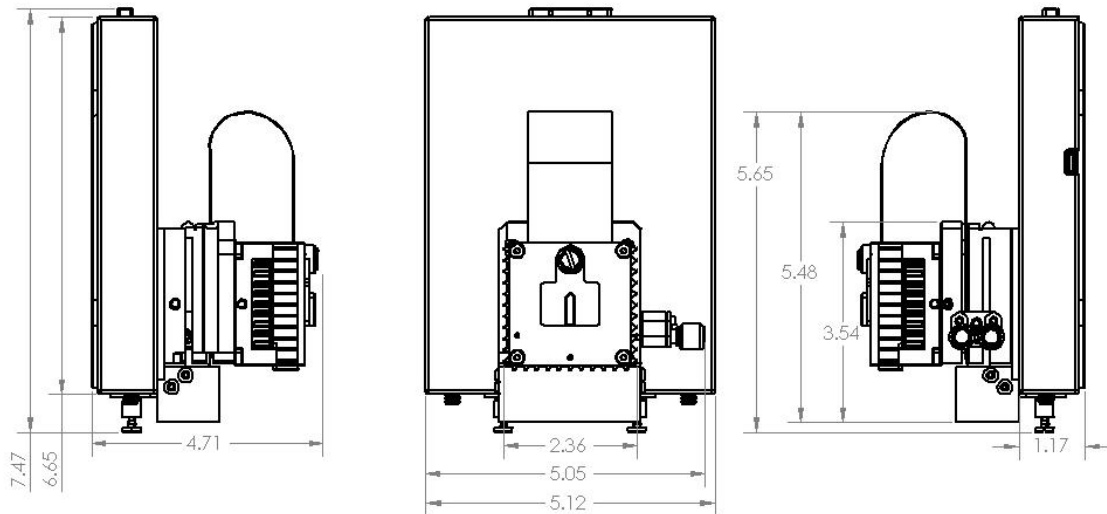


Figure 31 Front and side views of 1920 SLM Optical Head. Dimensions are in inches.

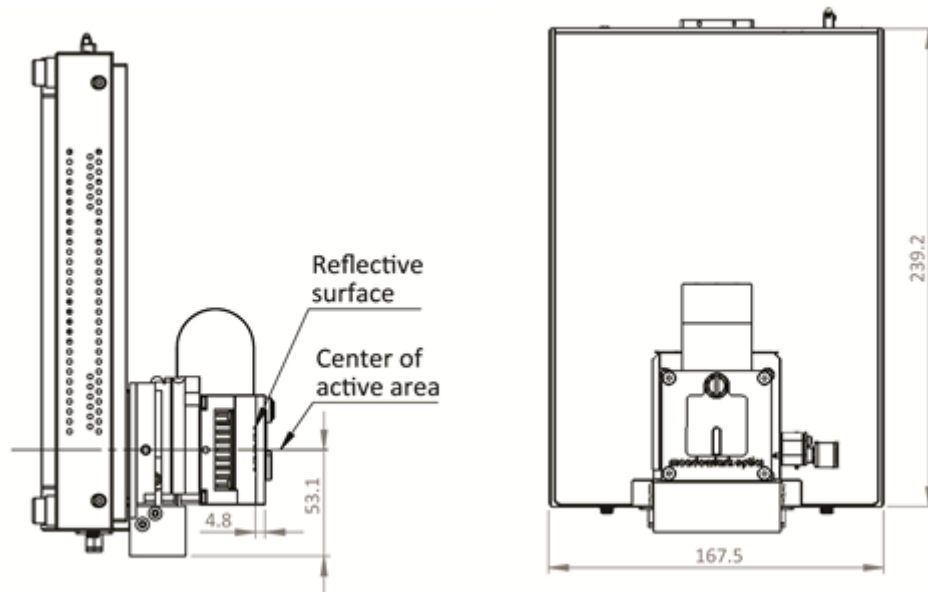


Figure 32 Front and side views of 1024 SLM Optical Head. Dimensions are in inches.

6 SLM care and cleaning

CAUTION

Failure to follow the recommendations included in this document may violate your warranty. Always ensure all personnel involved in the handling of your SLMs have appropriate tools, equipment, and training prior to cleaning any SLM.

Never use acetone to clean your SLM. Acetone will cause irreparable damage to your SLM.

Never use pressurized air or nitrogen to clean your SLM. Pressurized air or nitrogen will destroy the bond wires that provide the electrical interface to the SLM.

The SLM is shipped with a rotating aperture cover covering the SLM. This aperture, or a similar aperture, should be kept in place while the SLM is not in use to reduce the buildup of dust and debris on the SLM coverglass. When removing the aperture cover, or while the SLM coverglass is exposed, care should be taken to avoid any contact with the coverglass.

If dust or debris does contaminate the coverglass, gently remove them with a methanol soaked lens tissue. Take a small lens tissue or cleanroom wipe and soak a corner of it with methanol. Let the methanol evaporate until the lens tissue is just damp then gently wipe off the particles with the corner of the wipe. For more stubborn debris the methanol drag can be used. Fold a lens tissue several times until it is just small enough to fit inside of the SLM aperture. Soak the folded edge with methanol and let it evaporate until just damp. Line up the folded edge of the lens tissue with the edge of the glass, and drag the tissue straight across the glass. Never wipe the glass more than once with the same lens tissue as it will merely transfer dirt back onto the coverglass. Repeat as necessary.

If there is any dirt or contamination that cannot be removed, please contact the factory for additional cleaning instructions.

7 Troubleshooting

7.1 Image appears incorrect

When loading images to the SLM, make sure the image files are 1024 x 1024 8-bit bitmap images for the 1024 SLM, or 1920 x 1152 8-bit bitmap images for the 1920 SLM. If the image dimensions are incorrect, it will not be displayed properly on the SLM.

7.2 LEDs on the 1920x1152 or 1024x1024 PCIe hardware are blinking

There is dual color LED on the front panel of the PCIe hardware, which indicates various states:

LED behaviour	State
RED	During initial power up, if the red LED is on then external power is present but one of the internal regulators is faulty.
AMBER	FPGA not configured
RED blinking	Temperature alarm, FPGA temperature is above 90C. <i>Note: the alarm is removed when FPGA cools down to 85C</i>
AMBER blinking	LCOS not in sync, for example LCOS disconnected or faulty
GREEN blinking	PCIe not in sync, for example PCIe cable disconnected
GREEN	Normal operation, PCIe is in sync, LCOS is in sync

Thank you for purchasing a Meadowlark Optics Spatial Light Modulator.

For additional product and company information, please contact:

*Meadowlark Optics, Inc.
5964 Iris Parkway
P.O. Box 1000
Frederick, CO 80530*

*Telephone: 303-833-4333
Fax: 303-833-4335
Website: www.meadowlark.com*

Please feel free to contact us with any questions you may have as well as to leave feedback about your device.

For questions regarding customer support for Meadowlark SLM products, please contact us by telephone or by e-mailing slmsupport@meadowlark.com

For questions regarding purchasing and pricing of additional Meadowlark SLM products, please contact us by telephone or by e-mailing sales@meadowlark.com